Binary Search in Competitive Programming

A Pattern-Theoretic Treatment

Advanced Reference and Pattern Catalogue

Foundations · Core Patterns · Worked Examples

Department of Competitive Programming Notes

October 20, 2026

Contents

Preface **v**

I Foundations and Pattern Recognition **1**

1 Foundations and Formalism **2**
1.1 Monotone Predicates and the Search Space 2
1.2 The Algorithm as a Loop Invariant 3
1.3 The Halving Lemma and Invariant Discipline 4
1.4 Reference Implementation 4
1.5 Two Conventional Specialisations 5
1.5.1 Lower bound on a sorted array 5
1.5.2 Upper bound on a sorted array 5
1.6 Chapter Summary 5

2 Pattern Identification Framework **6**
2.1 The Four-Question Decision Procedure 6
2.2 Signal Words in Problem Statements 6
2.3 The Monotonicity Test 7
2.4 Predicate Design Principles 8
2.5 Structural Cues for the Advanced Patterns 8
2.6 A Decision Flowchart 8
2.7 Anti-Patterns: When Not to Binary Search 9
2.8 Chapter Summary 10

II Core Patterns **11**

3 Basic Search Patterns **12**
3.1 Lower and Upper Bound, Formally 12
3.2 Equal Range and Count 13
3.3 Worked Example: Find First and Last Position 13
3.4 Variations and Pitfalls 14
3.4.1 Search in a rotated sorted array 14
3.4.2 Search in a sorted matrix 14
3.4.3 Pitfall: integer overflow in mid 14
3.4.4 Pitfall: infinite loop with closed-interval mid 14
3.5 Chapter Summary 15

4 Binary Search on the Answer **16**
4.1 The Pattern in One Sentence . . . 16
4.2 Designing the Predicate . . . 16
4.3 Worked Example: Aggressive Cows (SPOJ) . . . 16
4.4 Worked Example: Split Array Largest Sum . . . 18
4.5 A Catalogue of BSA Sub-Patterns . . . 19
4.5.1 Minimise-the-maximum . . . 19
4.5.2 Maximise-the-minimum . . . 19
4.5.3 Smallest-k family . . . 19
4.5.4 Smallest-X-with-property . . . 19
4.6 When the Predicate is Itself a Search . . . 19
4.7 Chapter Summary . . . 20

5 Real-Valued Binary Search **21**
5.1 Termination: Fixed Iteration Count versus EPS . . . 21
5.2 Choosing the Iteration Count . . . 21
5.3 Reference Implementation . . . 22
5.4 Worked Example: Computing the Cube Root . . . 22
5.5 Worked Example: Maximum Average Subarray II . . . 22
5.6 Pitfalls Specific to Real-Valued Search . . . 23
5.6.1 NaN propagation . . . 23
5.6.2 Comparing with ε . . . 23
5.6.3 Range too wide . . . 24
5.7 Chapter Summary . . . 24

6 Ternary Search on Unimodal Functions **25**
6.1 The Unimodal Property . . . 25
6.2 Discrete versus Continuous Ternary Search . . . 25
6.3 Reference Implementation . . . 26
6.4 Worked Example: Minimising the Sum of Distances . . . 26
6.5 When Ternary Search Fails . . . 26
6.6 Golden-Section Search . . . 27
6.7 Chapter Summary . . . 27

7 Binary Search on Implicit Arrays **28**
7.1 The Pattern . . . 28
7.2 Worked Example: K-th Smallest in a Sorted Matrix . . . 28
7.3 Worked Example: Median of Two Sorted Arrays . . . 29
7.4 Worked Example: K-th Smallest Pair Distance . . . 30
7.5 A Catalogue of Counting Procedures . . . 30
7.6 Chapter Summary . . . 31

8 Parallel Binary Search **32**
8.1 When PBS Applies . . . 32
8.2 The Algorithm . . . 33
8.3 Worked Example: Dynamic Connectivity Offline . . . 33
8.4 Worked Example: Meteors (SPOJ) . . . 35
8.5 PBS with Rollback . . . 35
8.6 Chapter Summary . . . 35

9 Binary Lifting **36**
9.1 The Pattern . . . 36
9.2 Reference Implementation . . . 36
9.3 Worked Example: LCA on a Tree . . . 37
9.4 Worked Example: Functional Graph Cycle Detection . . . 38
9.5 Variations . . . 38
9.5.1 Sparse tables for RMQ . . . 38
9.5.2 Heavy-light decomposition . . . 38
9.6 Chapter Summary . . . 38

10 Divide and Conquer DP Optimisation **39**
10.1 The Setting . . . 39
10.2 The Quadrangle Inequality . . . 39
10.3 The D&C DP Algorithm . . . 40
10.4 Worked Example: Post Office Problem . . . 40
10.5 Variations and Related Optimisations . . . 42
10.5.1 Knuth optimisation . . . 42
10.5.2 Aliens (Lagrangian relaxation) . . . 42
10.5.3 Convex hull trick . . . 42
10.6 Chapter Summary . . . 42

III Practice and Pitfalls **43**

11 Common Bugs and Pitfalls **44**
11.1 Bug 1: Integer Overflow in Midpoint . . . 44
11.2 Bug 2: Infinite Loop with Closed-Interval Mid . . . 44
11.3 Bug 3: Wrong Sentinel at Boundaries . . . 45
11.4 Bug 4: Wrong Predicate Direction . . . 46
11.5 Bug 5: Off-By-One in Closed-Interval Update . . . 46
11.6 Bug 6: Real-Valued EPS Misuse . . . 46
11.7 Bug 7: Returning the Wrong Variable . . . 47
11.8 Bug 8: Predicate Mutates State . . . 47
11.9 Bug 9: Closed-Interval with Empty Range . . . 48
11.10Bug 10: Wrong Search Range . . . 48
11.11A Pre-Submission Checklist . . . 48
11.12Chapter Summary . . . 49

12 Additional Worked Examples **50**
12.1 Example 1: Koko Eating Bananas (LeetCode 875) . . . 50
12.2 Example 2: Capacity To Ship Packages (LeetCode 1011) . . . 51
12.3 Example 3: Pie (POJ 3122) . . . 52
12.4 Example 4: Allocate Mailboxes (LeetCode 1478) . . . 52
12.5 Example 5: Magnetic Force Between Two Balls (LeetCode 1552) . . . 53
12.6 Example 6: Heaters (LeetCode 475) . . . 54
12.7 Chapter Summary . . . 54

A Code Templates Reference **56**
A.1 Open-Interval Integer Binary Search . . . 56
A.2 Lower and Upper Bound . . . 56
A.3 Binary Search on the Answer . . . 57
A.4 Real-Valued Binary Search . . . 57

A.5 Ternary Search 57
A.6 Parallel Binary Search Skeleton 58
A.7 Binary Lifting (LCA) 58
A.8 Divide and Conquer DP 59
A.9 Pre-Submission Checklist 59

Preface

Binary search is, on its surface, the most elementary algorithm a competitive programmer learns. A sorted array, a comparison, and a halving step—at most a dozen lines of code. Yet behind this elementary facade lies one of the deepest and most frequently underestimated toolkits in the entire competitive programming discipline. Problems that have nothing to do with sorted arrays, that on first reading look like dynamic programming, graph theory, or geometry, routinely collapse into a handful of binary searches once the right predicate is formulated. The gap between the textbook presentation—"find an element in a sorted array"—and the competition-ready presentation—"monotone predicate inversion over arbitrary search spaces"—is precisely the gap this document is written to close.

This text assumes the reader is already comfortable with the *mechanics* of binary search. We will not spend paragraphs explaining what a sorted array is. Instead, we treat binary search as a *pattern family*—a collection of related algorithmic templates that all share a common mathematical core but differ radically in how they are applied. The unifying thread is the notion of a *monotone predicate*: a Boolean function p on a totally ordered search space S such that once p becomes true at some $x \in S$, it remains true for all larger elements. Every binary search—from `lower_bound` on an array to parallel binary search on a dynamic connectivity forest—is, at its core, an algorithm for inverting such a predicate in $\mathcal{O}(\log |S|)$ evaluations of p.

The document is organised into three parts. Part I develops the formal framework: predicate monotonicity, the invariant-based proof of correctness, and a decision procedure for *recognising* a binary search problem in the wild. This last contribution—a pattern-identification framework—is, in our experience, the single most valuable skill the advanced contestant can develop. Most contestants can implement binary search; far fewer can glance at a problem statement and correctly diagnose which binary search variant applies.

Part II catalogues the eight pattern families that recur in competitive programming: basic search on sorted arrays, binary search on the answer, real-valued binary search, ternary search on unimodal functions, binary search on implicit arrays, parallel binary search, binary lifting, and divide-and-conquer dynamic programming optimisation. Each chapter presents the pattern formally, gives a reference implementation in C++, walks through at least one worked example, and discusses the failure modes that distinguish a correct solution from a waivable one.

Part III catalogues the common bugs that cause otherwise-correct solutions to fail on edge cases—off-by-one errors, integer overflow in mid computation, infinite loops caused by predicate boundary confusion—and then provides a final batch of fully worked problems with complete solutions.

A note on conventions. Theorem environments are numbered per chapter; theorems are typeset in italic, definitions in roman, remarks in roman. Code is presented in `listings`-formatted C++ with line numbers. Throughout, we use the typedef `typedef long long ll;` and assume `const int MOD = 1e9+7;` where modular arithmetic is needed. Loop indices are half-open intervals wherever practical, and we treat the search space as the half-open interval

$[\mathsf{lo}, \mathsf{hi})$ unless explicitly noted otherwise.

Part I

Foundations and Pattern Recognition

Chapter 1

Foundations and Formalism

Binary search, in its most general form, is an algorithm for *inverting a monotone predicate over a totally ordered search space*. Stripped of all the accidents of presentation—the sorted array, the integer indices, the comparison operator—the algorithm is a single loop that maintains a candidate interval and halves it on each iteration. The purpose of this chapter is to make that definition precise, prove the algorithm correct via a loop invariant, and isolate the conditions under which it applies.

1.1 Monotone Predicates and the Search Space

Definition 1.1 (Search space)**.** A *search space* is a finite or continuous totally ordered set S equipped with a notion of *betweenness*; that is, for any two elements $a, b \in S$ with $a \le b$, the truncated suborder $[a, b]_S = \{x \in S : a \le x \le b\}$ admits a midpoint operation that produces, in $\mathcal{O}(1)$ time, an element $\mathsf{mid} \in [a, b]_S$ whose removal splits $[a, b]_S$ into two subsets of strictly smaller size.

In practice, the search space will be one of three kinds: a finite set of integers $\{0, 1, \ldots, n-1\}$ (discrete integer search); a finite but enormous range such as $\{0, 1, \ldots, 10^{18}\}$ (answering a question whose answer lives in a large integer range); or a real interval $[a, b] \subset \mathbb{R}$ (real-valued binary search). The midpoint operation is, respectively, the integer average $\lfloor (a + b)/2 \rfloor$, the careful integer average $a + (b - a)/2$ that avoids overflow, and the floating-point midpoint $(a + b)/2$.

Definition 1.2 (Monotone predicate)**.** Let S be a search space. A predicate $p : S \to \{\text{false}, \text{true}\}$ is *monotone increasing* if for all $a \le b$ in S,

$$p(a) = \text{true} \implies p(b) = \text{true}.$$

Equivalently, the truth set $T = \{x \in S : p(x) = \text{true}\}$ is upward-closed in S. The predicate is *monotone decreasing* if $\neg p$ is monotone increasing. We say a problem admits a binary search solution precisely when its feasibility predicate is monotone in this sense.

The reader should fix this definition in mind. Almost every bug in a binary search solution traces back to a predicate that is not, in fact, monotone—or to a search space that does not admit a midpoint operation. We will return to this in Chapter 2 when we develop the pattern-identification framework.

Example 1.3 (Monotonicity is not always obvious)**.** Consider the problem "given an array of n positive integers, partition it into k contiguous subarrays so as to minimise the maximum subarray sum". The natural predicate to formulate is "*can* the array be partitioned into k subarrays each with sum at most X?" Call this $p(X)$. Monotonicity is immediate once noticed:

if $p(X)$ holds, then $p(X')$ holds for every $X' \geq X$, because the same partition remains feasible. The search space is the integer range $[\max a_i, \sum a_i]$, and the answer is the smallest X with $p(X)$ true. Yet many contestants fail to see the monotonicity on first reading and attempt a $\mathcal{O}(n^2k)$ DP instead.

1.2 The Algorithm as a Loop Invariant

With Definitions 1.1 and 1.2 in hand, we can state the algorithm in its most abstract form. The input is a search space S, two endpoints $\mathsf{lo} \leq \mathsf{hi}$ in S with $[\mathsf{lo}, \mathsf{hi}] \subseteq S$, and a monotone increasing predicate p. The output is the smallest element $x^* \in [\mathsf{lo}, \mathsf{hi}]$ such that $p(x^*)$ is true, or a sentinel value $\perp$ if no such x exists.

Algorithm 1 Abstract binary search (integer setting)

```
1: lo ← L;  hi ← R                    ▷ Maintain invariant: p(lo) = false, p(hi) = true
2: while hi − lo > 1 do
3:     mid ← lo + (hi − lo)/2
4:     if p(mid) then
5:         hi ← mid
6:     else
7:         lo ← mid
8:     end if
9: end while
10: return hi
```

The choice of invariant is critical and slightly non-obvious. Algorithm 1 maintains, at the top of each iteration, the precondition

$$p(lo) = \text{false}, \qquad p(hi) = \text{true},$$

with the convention that an initial search range $[L, R]$ has been padded on both sides: $L - 1$ is treated as a virtual element on which p is false, and $R + 1$ as a virtual element on which p is true. If p is true on every element of $[L, R]$, the algorithm returns L; if p is false on every element, it returns $R + 1$ (the sentinel $\perp$).

Theorem 1.4 (Correctness of binary search)**.** *Let S be a discrete search space, $[L, R] \subseteq S$, and p a monotone increasing predicate on S. Suppose the* padded precondition *holds: $p(L - 1) =$ false and $p(R + 1) =$ true. Then Algorithm 1 terminates after at most $\lceil \log_2(R - L + 2) \rceil$ iterations and returns the unique element $x^* \in [L, R + 1]$ satisfying*

$$p(x^* - 1) = \textit{false}, \qquad p(x^*) = \textit{true}.$$

Proof. The proof proceeds in three parts: *invariant maintenance*, *termination*, and *partial correctness*.

Invariant. We claim that at the start of each iteration of the while-loop in line 3, the invariant $\mathcal{I} := (p(lo) = \text{false}) \wedge (p(hi) = \text{true})$ holds. At initialisation, $lo = L - 1$ and $hi = R + 1$ (interpreting $L - 1$ and $R + 1$ as the virtual padded elements), so $\mathcal{I}$ holds by the padded precondition. For the inductive step, suppose $\mathcal{I}$ holds at the top of some iteration; we show it holds at the top of the next. The midpoint mid satisfies $lo < \mathsf{mid} < hi$, so by totality of the order and the strict inequality, mid lies strictly inside the current interval. Two cases arise. If $p(\mathsf{mid}) = \text{true}$, the algorithm sets $hi \leftarrow \mathsf{mid}$; the new pair is (lo, mid), and $p(\mathsf{mid}) = \text{true}$ by assumption while $p(lo) = \text{false}$ by the inductive hypothesis, so $\mathcal{I}$ is preserved. If $p(\mathsf{mid}) = \text{false}$, the algorithm sets $lo \leftarrow \mathsf{mid}$; the new pair is (mid, hi), and $p(\mathsf{mid}) = \text{false}$ by assumption while $p(hi) = \text{true}$ by hypothesis. Either way $\mathcal{I}$ holds.

Termination. Each iteration replaces the interval (lo, hi) of length $d = hi - lo$ by either (lo, mid) or (mid, hi), both of length strictly less than d. In the integer setting with $\mathsf{mid} = lo + (hi - lo)/2$, the new length is at most $\lceil d/2 \rceil - 1 < d$ for $d \geq 2$. Hence the loop terminates when $hi - lo = 1$, which occurs after at most $\lceil \log_2(R - L + 2) \rceil$ halvings of the initial length $R - L + 2$.

Partial correctness. Upon termination, $hi - lo = 1$ and $\mathcal{I}$ holds. The element hi satisfies $p(hi) = \text{true}$ (by $\mathcal{I}$) and $p(hi - 1) = p(lo) = \text{false}$ (by $\mathcal{I}$ again). Since $hi \in [L, R + 1]$ (the invariant confines it to this range) and the predicate is monotone, hi is the unique element satisfying both conditions. Setting $x^* = hi$ completes the proof. □

Corollary 1.5 (Complexity)**.** *Algorithm 1 performs at most* $\lceil \log_2(R - L + 2) \rceil$ *evaluations of the predicate* p*. If each evaluation costs* $T(n)$*, the total cost is* $\mathcal{O}(T(n) \log(R - L))$*.*

This complexity bound is the single most important fact about binary search. It tells us that we can afford predicate evaluations costing up to $\mathcal{O}(n)$ if the search range is polynomial in n, and still beat a $\mathcal{O}(n^2)$ brute force. The interplay between the predicate cost $T(n)$ and the search range log-factor is what makes binary search on the answer such a powerful technique.

1.3 The Halving Lemma and Invariant Discipline

Lemma 1.6 (Halving lemma)**.** *For any initial interval* $[L, R]$ *of length* $n = R - L + 1$*, after* k *iterations of binary search the surviving interval has length at most* $\lceil n/2^k \rceil$*. In particular,* $\lceil \log_2 n \rceil$ *iterations suffice to reduce the interval to length* 1*.*

The halving lemma is the engine of every binary-search variant. The practical consequence is that even enormous search ranges—say $[1, 10^{18}]$—require only 60 iterations of binary search, which is well within any time limit. This is why binary search on the answer is so effective for problems whose answer space is exponential in n.

Key Idea

The **invariant discipline** is the practice of explicitly writing down the precondition $\mathcal{I}$ at the top of your binary search loop, before writing the loop body. The two most common conventions are:

- **Open-interval style** (used in Algorithm 1): $p(lo) = \text{false}$, $p(hi) = \text{true}$, return hi. Loop condition $hi - lo > 1$.
- **Closed-interval style**: p is false on $[L, lo)$ and true on $[hi, R]$. Loop condition $lo \leq hi$, return lo (or $hi + 1$).

Pick *one* convention and use it for every binary search you write. Mixing conventions is the single most common source of off-by-one bugs. We use the open-interval style throughout this document.

1.4 Reference Implementation

Listing 1.1 gives a reference implementation of the open-interval binary search in C++. It is parameterised on the predicate p, which is passed as a callable. The same template applies whether the search space is an array index range, an answer space, or a real-valued interval (with the loop condition adjusted to a fixed iteration count for the latter).

```
1 // Returns the smallest x in [lo, hi] with p(x) == true.
2 // Precondition: p(hi) == true (caller's responsibility).
3 // If no such x exists in [lo, hi], behaviour depends on caller:
```

```
//   either extend hi with a virtual true, or return hi + 1 as sentinel.
template <class Pred>
long long first_true(long long lo, long long hi, Pred p) {
    hi++;  // virtual element on which p is true (postcondition)
    while (hi - lo > 1) {
        long long mid = lo + (hi - lo) / 2;
        if (p(mid)) hi = mid;
        else        lo = mid;
    }
    return hi;
}
```

Listing 1.1: Open-interval binary search template.

The virtual extension `hi++` on line 6 is the trick that removes the need for the caller to guarantee that p is true at the original `hi`. The caller passes the genuine search range $[lo, hi]$, and the function internally treats position $hi+1$ as a virtual element on which p is true. If the predicate is false on every element of $[lo, hi]$, the function returns $hi+1$ as a sentinel, and the caller can detect this case by comparing the return value against $hi+1$.

1.5 Two Conventional Specialisations

Most binary searches encountered in practice specialise the abstract template in one of two directions.

1.5.1 Lower bound on a sorted array

Given a sorted array $a[0..n-1]$ and a query value v, the *lower bound* is the smallest index i such that $a[i] \geq v$. The predicate is $p(i) = (a[i] \geq v)$; it is monotone increasing because a is sorted. The standard library `std::lower_bound` implements exactly this.

1.5.2 Upper bound on a sorted array

The *upper bound* is the smallest index i such that $a[i] > v$. The predicate is $p(i) = (a[i] > v)$, again monotone increasing. The pair (`lower_bound`, `upper_bound`) partitions the array into elements $< v$, $= v$, and $> v$.

The two functions are dual: `upper_bound(a, v)` equals `lower_bound(a, v+1)` when v is an integer. This duality is often the cleanest way to remember which one to use.

1.6 Chapter Summary

We have presented binary search as the inversion of a monotone predicate over a totally ordered search space, given a loop-invariant proof of correctness, and isolated the two ingredients—monotonicity of the predicate and totality of the order on the search space—that make the algorithm applicable. In Chapter 2 we turn from the formalism to the practical question of pattern recognition: how, given a problem statement, do we decide whether a binary search applies and which variant to use?

Chapter 2

Pattern Identification Framework

If you ask an experienced competitive programmer what distinguishes a binary search problem, you will receive a diffuse answer: "it's about minimising the maximum", "the answer is monotone in the input", "there's a threshold somewhere". Each of these hints is correct, but none is operational. The goal of this chapter is to give an *operational* decision procedure—a sequence of tests the solver can apply to a problem statement to determine, in at most a few minutes, whether binary search applies and which variant.

2.1 The Four-Question Decision Procedure

When you read a problem statement for the first time, ask yourself the following four questions in order. If all four can be answered affirmatively, the problem is a binary search problem.

Q1. Optimisation or feasibility? Is the problem asking for the minimum/maximum value of some quantity, or the minimum/maximum threshold at which some property becomes feasible?

Q2. Monotonicity of feasibility. Suppose we phrase the problem as "find the minimum X such that $\Phi(X)$ holds", where Φ is some Boolean predicate on the answer. Is Φ monotone increasing—i.e., does $\Phi(X) \Rightarrow \Phi(X+1)$? Equivalently: *does increasing X only ever make the problem easier?*

Q3. Efficient predicate evaluation. Given a candidate X, can we evaluate $\Phi(X)$ in time polynomial in n (typically $\mathcal{O}(n)$ or $\mathcal{O}(n \log n)$)? If yes, the total cost will be $\mathcal{O}(n \log \text{range})$ which is acceptable.

Q4. Bounded search range. Is the answer confined to a range whose log is acceptable? For integer answers, ranges up to 10^{18} are fine (60 iterations). For real answers, an iteration count of 60–100 gives double precision.

If the answer to Q1 is yes, you have a binary-search-on-answer problem (Chapter 4). If the answer to Q1 is no but the problem involves a sorted array and the search for an element or boundary, you have a basic search problem (Chapter 3). If the problem involves maximising/minimising a function that is unimodal, you have ternary search (Chapter 6). The remaining patterns—implicit array, parallel BS, binary lifting, D&C DP—are recognisable by their structural cues, discussed in §2.5.

2.2 Signal Words in Problem Statements

Certain phrases in problem statements are reliable signals that a binary search solution exists. Table 2.1 lists the most common ones, together with the pattern they typically indicate.

Signal phrase	**Interpretation**	**Pattern**
"minimise the maximum…"	Each unit of X relaxes a constraint; feasibility is monotone in X.	BS on answer
"maximise the minimum…"	Each unit of X tightens a constraint; feasibility is monotone in X.	BS on answer
"is it possible to… within k"	k is the search variable; feasibility is monotone in k.	BS on answer
"k-th smallest / k-th largest"	Search over value; count elements $\leq x$ to evaluate predicate.	Implicit array
"find the smallest X such that…"	Direct predicate inversion.	BS on answer
"if we can do X in time T, can we also do X'?"	Monotone feasibility transition.	BS on answer
"…real-valued…… within 10^{-6}"	Real search space; tolerance suggests EPS or iteration-count BS.	Real-valued BS
"…convex / concave / unimodal"	Single-peak structure; golden section or ternary search.	Ternary search
"offline queries; each asks when… happens"	Many independent monotone searches; parallelise.	Parallel BS
"k-th ancestor / k-th successor on a tree"	Binary decomposition of k.	Binary lifting
"LCA / distance / path on a static tree"	Precompute ancestors at power-of-two depths.	Binary lifting
"$1D/1D$ DP, transitions look like…"	Quadrangle inequality on cost; monotone decision point.	D&C DP

Table 2.1: Signal words and the patterns they typically indicate.

The signal-word table is heuristic, not algorithmic. A problem may use several of these phrases, or none of them. The four-question procedure of §2.1 remains the authoritative test. Use the signal words as a starting prompt, then verify with the four questions.

2.3 The Monotonicity Test

Question Q2 is the most frequently fumbled of the four. The right way to perform the monotonicity test is to write down the predicate $\Phi(X)$ explicitly, then ask: "if I know $\Phi(X)$ is true, can I prove $\Phi(X+1)$ is true without re-examining the input?"

Example 2.1 (Monotonicity check)**.** Consider the partition problem from Example 1.3. The predicate is

$\Phi(X) :=$ "the array can be partitioned into k contiguous subarrays each with sum at most X".

Suppose $\Phi(X)$ holds via some partition $\mathcal{P} = (P_1, \ldots, P_k)$. We claim $\Phi(X+1)$ holds: the *same* partition $\mathcal{P}$ witnesses it, because each P_i has sum at most $X < X+1$. Hence Φ is monotone increasing.

Example 2.2 (Failure of monotonicity)**.** Consider "find the smallest X such that the sum of array elements in positions $1, 2, \ldots, X$ is divisible by 7". The predicate $\Phi(X) := (7 \mid \sum_{i \leq X} a_i)$ is *not* monotone: divisibility flickers as X grows. Binary search does not apply; the correct solution is prefix sums with a hash map.

The monotonicity test catches a large fraction of misdiagnosed binary search attempts. *If you cannot prove* $\Phi(X) \Rightarrow \Phi(X+1)$ *in one sentence, do not binary search.*

2.4 Predicate Design Principles

The art of binary search lies less in the search itself than in the *design of the predicate*. Three principles guide this design.

P1. Make the predicate feasibility, not optimality. The predicate should be "can we achieve *at least / at most* X?", not "is X optimal?". The latter is rarely monotone; the former almost always is.

P2. Decouple predicate from search direction. Decide first whether you are minimising or maximising, then design the predicate so that *true* answers correspond to "feasible at this threshold". For a minimisation problem, the answer is the smallest X with $\Phi(X)$ true. For a maximisation problem, you can either negate the predicate (and search for the smallest X with $\Phi(X)$ false) or run the search in the opposite direction.

P3. Pad the boundaries. Always extend the search range with one virtual element on each side: a virtual *false* below the lower bound (representing "impossibly small") and a virtual *true* above the upper bound (representing "trivially feasible"). This eliminates special cases in the loop and is what the `hi++` trick in Listing 1.1 accomplishes.

2.5 Structural Cues for the Advanced Patterns

The four-question procedure handles the binary-search-on-answer family. The remaining patterns are recognised by their structural cues:

Implicit array (Ch. 7) The problem mentions "k-th smallest/largest" but the underlying multiset is too large to materialise. The cue is that the elements can be *counted* without being enumerated—e.g., a sorted matrix where counting elements $\leq x$ takes $\mathcal{O}(n)$.

Parallel BS (Ch. 8) The problem describes many queries, each asking "when does event E_i first occur?", and the events can be ordered in time. The cue is offline queries over a temporal or chronological process. Instead of binary searching each query independently (which would re-process the same events many times), parallel BS processes all queries' mid-points simultaneously in $\mathcal{O}(\log T)$ rounds.

Binary lifting (Ch. 9) The problem involves *iterated* transitions: "apply operation f to x a total of k times". The cue is a large k (e.g., $k \leq 10^{18}$) and a static transition function f. Binary lifting precomputes $f^{(2^i)}(x)$ for each i, then composes these by reading the bits of k.

D&C DP (Ch. 10) The problem is a 1D/1D dynamic program—$\mathcal{O}(n)$ states, each with $\mathcal{O}(n)$ candidate predecessors—and the cost function satisfies the *quadrangle inequality*. The cue is a DP of the form $dp[i] = \min_{j<i}\{dp[j] + w(j,i)\}$ where w is a cost function with a Monge-like property.

2.6 A Decision Flowchart

Figure 2.1 condenses the four-question procedure and the structural cues into a single decision flowchart. The flowchart is intended to be read top-to-bottom: start at the root, answer each question, and arrive at one of the eight pattern leaves (or at the "no binary search applies" exit).

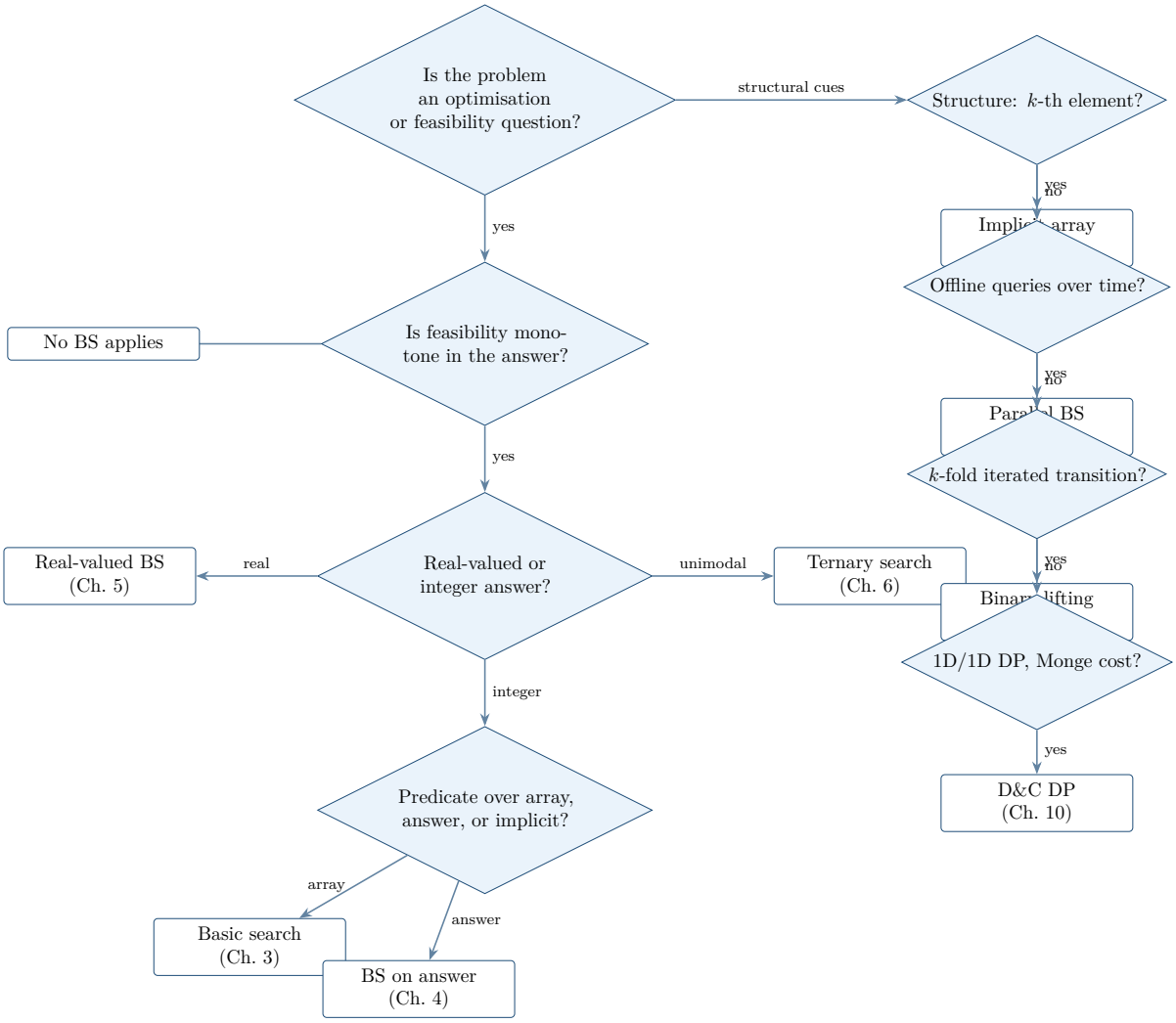


Figure 2.1: Decision flowchart for binary search pattern identification. Follow the yes/no answers from the root to a leaf; the leaf indicates the chapter containing the relevant pattern.

2.7 Anti-Patterns: When Not to Binary Search

Conspicuously absent from the flowchart is the question "does the problem have a sorted array?" A sorted array is *not* a reliable cue for binary search. Many problems with sorted-looking inputs are best solved with two pointers, monotone stacks, or greedy methods. Conversely, many problems with *unsorted* inputs admit binary search on the answer. The presence or absence of a sorted array is therefore irrelevant to the pattern-identification procedure.

Warning

The most common misdiagnosis is to assume "binary search" from the phrase "sorted array" and then attempt to find an element by bisection. If the problem is asking for an *aggregation* (e.g., "how many pairs have sum $\leq k$?"), the correct tool is usually *two pointers* ($\mathcal{O}(n)$), not binary search ($\mathcal{O}(n \log n)$). Two pointers should be your first thought whenever the sortedness of the input is the only signal.

2.8 Chapter Summary

The pattern-identification framework rests on three pieces: a four-question decision procedure (Q1–Q4) for the binary-search-on-answer family, a catalogue of signal words that prompt specific patterns, and a structural-cue list for the four advanced patterns (implicit array, parallel BS, binary lifting, D&C DP) that the four-question procedure does not directly cover. With this framework in hand, the reader is equipped to diagnose problems; the remaining chapters of Part II develop each pattern in depth.

Part II

Core Patterns

Chapter 3

Basic Search Patterns

The basic search pattern is the specialisation of Algorithm 1 to the case where the search space is the index range of a sorted array. Despite its simplicity, this pattern deserves a chapter of its own for three reasons. First, the standard library functions `std::lower_bound` and `std::upper_bound` are the workhorses of competitive programming and the reader should know their exact semantics cold. Second, many "hard" problems reduce to a sequence of basic searches composed with two-pointers or sweep-line passes. Third, the basic search is the right place to internalise the open-interval convention before moving on to the more elaborate variants.

3.1 Lower and Upper Bound, Formally

Let $a[0..n-1]$ be a sorted array of comparable elements, and let v be a query value.

Definition 3.1 (Lower bound)**.** The *lower bound* of v in a is the smallest index $i \in [0, n]$ such that $a[i] \geq v$, with the convention $a[n] = +\infty$.

Definition 3.2 (Upper bound)**.** The *upper bound* of v in a is the smallest index $i \in [0, n]$ such that $a[i] > v$, with the convention $a[n] = +\infty$.

The two functions are dual: `upper_bound(a, v)` equals `lower_bound(a, v+1)` when v is integral. They partition the array into three regions—elements $< v$, elements $= v$, and elements $> v$—and the count of v in a is exactly $\textsc{upper_bound}(v) - \textsc{lower_bound}(v)$.

```cpp
// Returns smallest i in [0, n] with a[i] >= v.
// (a[n] is treated as +infinity, so the answer is always well-defined.)
int lower_bound_idx(const vector<int>& a, int v) {
    int lo = -1, hi = (int)a.size();
    // invariant: a[lo] < v  (lo = -1 means a[-1] = -infinity)
    // invariant: a[hi] >= v  (hi = n   means a[n]  = +infinity)
    while (hi - lo > 1) {
        int mid = lo + (hi - lo) / 2;
        if (a[mid] >= v) hi = mid;
        else             lo = mid;
    }
    return hi;
}

// Returns smallest i in [0, n] with a[i] > v.
int upper_bound_idx(const vector<int>& a, int v) {
    int lo = -1, hi = (int)a.size();
    // invariant: a[lo] <= v  (lo = -1 means -infinity)
    // invariant: a[hi] >  v  (hi = n   means +infinity)
    while (hi - lo > 1) {
```

```
        int mid = lo + (hi - lo) / 2;
        if (a[mid] > v) hi = mid;
        else            lo = mid;
    }
    return hi;
}
```

Listing 3.1: Open-interval implementations of lower and upper bound. Note the use of `n+1` as the virtual true element.

The two implementations are byte-for-byte identical except for the comparison operator. This is the duality between lower and upper bound made concrete.

3.2 Equal Range and Count

The C++ standard library provides `std::equal_range`, which returns a pair of iterators bounding the range of elements equal to v. Implemented directly, it is simply a pair of `lower_bound` and `upper_bound` calls, costing $\mathcal{O}(2 \log n)$ comparisons. The count of v in a is the difference of the two iterators.

```
#include <algorithm>
// Number of elements equal to v in a sorted array.
long long count_eq(const vector<int>& a, int v) {
    auto lo = lower_bound(a.begin(), a.end(), v);
    auto hi = upper_bound(a.begin(), a.end(), v);
    return hi - lo;
}

// Equivalent: equal_range returns the pair (lo, hi) in one call.
auto [lo, hi] = equal_range(a.begin(), a.end(), v);
```

Listing 3.2: Counting occurrences and finding equal-range with the standard library.

3.3 Worked Example: Find First and Last Position

We illustrate the basic search pattern with LeetCode 34, "Find First and Last Position of Element in Sorted Array". The problem asks, given a sorted array and a target value, to return the indices of the first and last occurrence of the target, or $[-1, -1]$ if the target is absent.

Problem 3.3 (LeetCode 34)**.** Given an array of integers `nums` sorted in non-decreasing order, and an integer `target`, find the starting and ending position of `target` in `nums`. Return $[-1, -1]$ if `target` is not found.

Pattern identification. The array is sorted; the problem asks for two boundary positions. By Table 2.1, "find the first occurrence" is a lower bound query, and "find the last occurrence" is one less than an upper bound query. The four questions confirm: Q1 yes (we are locating a threshold), Q2 yes (monotone—once $a[i] \geq$ target, all larger indices also satisfy this), Q3 yes (predicate evaluation is $\mathcal{O}(1)$), Q4 yes (range is $[0, n]$).

Approach. Compute $\ell = \textsc{lower_bound}(target)$ and $u = \textsc{upper_bound}(target)$. If $\ell = u$ then the target is absent; return $[-1, -1]$. Otherwise return $[\ell, u - 1]$.

Complexity. $\mathcal{O}(\log n)$ time, $\mathcal{O}(1)$ space.

```
class Solution {
public:
    vector<int> searchRange(vector<int>& nums, int target) {
        int n = (int)nums.size();
        int lo = (int)(lower_bound(nums.begin(), nums.end(), target)
```

```
                        - nums.begin());
        int hi = (int)(upper_bound(nums.begin(), nums.end(), target)
                        - nums.begin());
        if (lo == hi) return {-1, -1};
        return {lo, hi - 1};
    }
};
```

Listing 3.3: Solution to LeetCode 34.

3.4 Variations and Pitfalls

The basic search pattern admits several variations that frequently appear as substeps of harder problems. We catalogue the most common ones.

3.4.1 Search in a rotated sorted array

A *rotated sorted array* is one in which the sorted sequence has been cut at some pivot and the two halves swapped: e.g., $[4, 5, 6, 7, 1, 2, 3]$. Binary search still works, but the comparison logic in the loop body must decide which half is properly sorted before applying the standard check. The key insight is that at least one of the two halves $[lo, \mathsf{mid}]$ or $[\mathsf{mid}, hi]$ is always properly sorted; we identify which, then check whether the target lies in the sorted half before recursing.

3.4.2 Search in a sorted matrix

A matrix is sorted in the sense that each row and each column is sorted in non-decreasing order. The search problem is to find whether a target value exists. The standard $\mathcal{O}(m+n)$ solution starts at the top-right corner and moves left or down based on the comparison with the target—this is, in spirit, a binary search where each step discards an entire row or column. We treat the harder "k-th smallest in a sorted matrix" problem in Chapter 7.

3.4.3 Pitfall: integer overflow in mid

The naive midpoint computation `(lo + hi)/ 2` overflows when `lo + hi` exceeds the maximum value of the type. With `long long`, this is rarely an issue, but with `int` and large arrays indexed near $2 \cdot 10^9$, it is a real bug. The fix is the identity

$$\mathsf{mid} = \mathsf{lo} + \frac{\mathsf{hi} - \mathsf{lo}}{2},$$

which cannot overflow because the difference $\mathsf{hi} - \mathsf{lo}$ is bounded by the array size. All templates in this document use this form.

3.4.4 Pitfall: infinite loop with closed-interval mid

In the closed-interval convention with loop condition `lo <=>hi` and midpoint `mid = (lo + hi)/ 2`, the update `lo = mid` (without $+1$) can cause an infinite loop when $\mathsf{hi} = \mathsf{lo} + 1$, because `mid` equals `lo` and the interval does not shrink. The fix is to use `mid = lo + (hi - lo + 1)/ 2` whenever the update is `lo = mid`, or to switch to the open-interval convention. We discuss this in detail in Chapter 11.

3.5 Chapter Summary

The basic search pattern is the open-interval binary search applied to a sorted array index range. The two essential operations are `lower_bound` and `upper_bound`, whose duality—upper is lower-with-v+1—captures their relationship cleanly. Most "find the boundary" problems reduce to one or two such calls. The pattern's value extends beyond its direct use: the open-interval discipline and the overflow-safe midpoint computation recur in every more elaborate variant.

Chapter 4

Binary Search on the Answer

Binary search on the answer—hereafter abbreviated *BSA*—is the workhorse pattern of competitive programming. The pattern arises whenever the problem asks for the optimal value of some quantity and the feasibility predicate over that quantity is monotone. BSA differs from basic search in two essential ways: the search space is no longer an array index range but an *answer range* (often enormous), and the predicate is no longer a single comparison but a sub-routine that may itself cost $\mathcal{O}(n)$ or $\mathcal{O}(n \log n)$ to evaluate.

4.1 The Pattern in One Sentence

Key Idea

BSA in one sentence. If the problem asks "find the minimum X such that $\Phi(X)$ holds" and Φ is monotone increasing in X, binary-search the smallest X in a suitable range with Φ as the predicate.

The four ingredients are: (i) a quantity to optimise, (ii) a predicate Φ expressing feasibility at a given threshold, (iii) a proof that Φ is monotone, and (iv) a search range $[L, R]$ within which the answer is guaranteed to lie. The answer is the smallest $X \in [L, R]$ with $\Phi(X)$ true, found by Algorithm 1.

4.2 Designing the Predicate

The predicate is the heart of a BSA solution. Three observations guide its design.

D1. The predicate is feasibility, not optimality. The predicate should answer "can we achieve at least/at most X?", not "is X optimal?". The latter is rarely monotone.

D2. The predicate is independent of the search. The predicate $\Phi(X)$ must be evaluable *without* knowing the eventual answer. It is a function of X and the input only; the search itself never appears inside Φ.

D3. The predicate cost dominates the total cost. If the predicate costs $T(n)$ and the search range has R distinct values, the total cost is $\mathcal{O}(T(n) \log R)$. The constant factor in $\log R$ is small ($\log_2 10^{18} \approx 60$), so even $\mathcal{O}(n^2)$ predicates can be acceptable for $n \leq 500$.

4.3 Worked Example: Aggressive Cows (SPOJ)

Problem 4.1 (Aggressive cows, SPOJ)**.** Farmer John has built N stalls at positions $x_1 < x_2 < \cdots < x_N$ on a line. He has C cows ($2 \leq C \leq N$) and wants to place them in stalls so

as to maximise the minimum distance between any two cows. Find this maximum possible minimum distance.

Pattern identification. The signal phrase "maximise the minimum" is a textbook BSA cue (Table 2.1). Q1 yes (maximising a minimum), Q2 yes (if a minimum distance d is achievable, so is every $d' < d$, by the same placement), Q3 yes (we can check feasibility in $\mathcal{O}(n)$ by a greedy left-to-right pass), Q4 yes (the range is $[1, x_N - x_1]$, which is at most 10^9).

Predicate. Define

$\Phi(d) :=$ "C cows can be placed such that every pair of adjacent placed cows is at distance $\geq d$".

Φ is monotone decreasing in d (larger d is harder). To make it monotone increasing as our framework requires, we either search in the opposite direction (largest d with $\Phi(d)$ true) or negate: $\Phi'(d) := \neg\Phi(d+1)$ is monotone increasing.

Predicate evaluation. Greedy: place the first cow at x_1. For each subsequent stall in order, place a cow there if its distance from the last placed cow is at least d. Count the total cows placed; $\Phi(d)$ is true iff the count is $\geq C$.

Search range. $L = 1$ (cows in adjacent stalls), $R = x_N - x_1$ (cows at the two extremes).

Complexity. $\mathcal{O}(n \log R) = \mathcal{O}(n \log(\max x))$.

```cpp
#include <bits/stdc++.h>
using namespace std;

bool can_place(const vector<int>& x, int C, long long d) {
    int placed = 1;
    long long last = x[0];
    for (int i = 1; i < (int)x.size(); i++) {
        if (x[i] - last >= d) {
            placed++;
            last = x[i];
            if (placed >= C) return true;
        }
    }
    return placed >= C;
}

int main() {
    int t; cin >> t;
    while (t--) {
        int N, C; cin >> N >> C;
        vector<int> x(N);
        for (int& v : x) cin >> v;
        sort(x.begin(), x.end());
        // Largest d such that can_place(d) is true.
        // Equivalently: first d such that can_place(d) is FALSE, minus 1.
        long long lo = 1, hi = (long long)x[N-1] - x[0] + 1;
        // Open interval: invariant can_place(lo)=true, can_place(hi)=false.
        // (hi is virtual-false: a distance larger than the spread.)
        while (hi - lo > 1) {
            long long mid = lo + (hi - lo) / 2;
            if (can_place(x, C, mid)) lo = mid;  // true -> raise lo
            else                      hi = mid;  // false -> lower hi
        }
        cout << lo << "\n";  // lo is the largest d with can_place(d)=true
    }
    return 0;
}
```

Listing 4.1: Solution to SPOJ Aggressive Cows.

Remark. Note the convention choice here. Because we want the *largest* d with $\Phi(d)$ true (and Φ is monotone decreasing), we maintain the invariant "`lo` is feasible, `hi` is infeasible" and return `lo`. This is the open-interval convention run in reverse. The alternative—negate the predicate to get a monotone increasing one and search for the first true—is equally valid; pick whichever is clearer for the problem at hand.

4.4 Worked Example: Split Array Largest Sum

Problem 4.2 (LeetCode 410)**.** Given an array `nums` of n non-negative integers and an integer k, partition the array into k non-empty contiguous subarrays. The *cost* of a partition is the largest subarray sum. Minimise the cost.

Pattern identification. "Minimise the maximum subarray sum" is a textbook BSA cue. Q1 yes (minimising a maximum), Q2 yes (if a maximum-sum M is achievable, so is every $M' \geq M$, by the same partition), Q3 yes (feasibility is a greedy left-to-right pass in $\mathcal{O}(n)$), Q4 yes (range is $[\max a_i, \sum a_i]$).

Predicate.

$$\Phi(M) := \text{"the array can be partitioned into } k \text{ subarrays each with sum} \leq M\text{"}.$$

Monotone increasing in M.

Predicate evaluation. Greedy: walk left to right, adding elements to the current subarray until adding the next would exceed M; then start a new subarray. Count the subarrays used. $\Phi(M)$ is true iff the count is $\leq k$. (If any single element exceeds M, the predicate is false immediately.)

Search range. $L = \max_i a_i$ (lower bound, since each subarray contains at least the max element), $R = \sum_i a_i$ (upper bound, since one subarray suffices).

Complexity. $\mathcal{O}(n \log(\sum a_i))$. For $n = 10^5$ and $\sum a_i = 10^9$, this is about $30 \cdot 10^5 = 3 \times 10^6$ operations—comfortably within time limits.

```cpp
class Solution {
public:
    int splitArray(vector<int>& nums, int k) {
        long long lo = 0, hi = 0;
        for (int v : nums) {
            lo = max(lo, (long long)v);
            hi += v;
        }
        // Open interval: lo infeasible-or-min-feasible,
        // hi feasible. We want smallest feasible.
        // Maintain: can_partition(lo - 1) = false, can_partition(hi) = true.
        // Equivalently: invariant (lo-1 infeasible, hi feasible).
        while (hi - lo > 1) {
            long long mid = lo + (hi - lo) / 2;
            if (can_partition(nums, k, mid)) hi = mid;
            else                             lo = mid;
        }
        return (int)hi;
    }
private:
    bool can_partition(const vector<int>& nums, int k, long long M) {
        int subs = 1;
        long long cur = 0;
        for (int v : nums) {
            if (cur + v > M) {
                subs++;
                cur = v;
```

```
                if (subs > k) return false;
            } else {
                cur += v;
            }
        }
        return true;
    }
};
```

Listing 4.2: Solution to LeetCode 410.

4.5 A Catalogue of BSA Sub-Patterns

Within the broad BSA family, certain sub-patterns recur often enough to deserve names. We catalogue the most common.

4.5.1 Minimise-the-maximum

The "minimise the maximum subarray sum / load / time / distance" family. The predicate is "can we partition / schedule / route such that no part exceeds X?". Greedy evaluation walks left-to-right or schedules first-fit-decreasing; the answer is the smallest X for which the greedy succeeds.

4.5.2 Maximise-the-minimum

The dual family: "maximise the minimum gap / capacity / quality". The Aggressive Cows problem of §4.3 is the paradigmatic example. The predicate is "can we choose / place such that every chosen pair / part achieves at least X?"; the answer is the largest X for which the predicate holds.

4.5.3 Smallest-k family

"Find the smallest k such that $\Phi(k)$ holds", where Φ counts some threshold-related property. Often $\Phi(k)$ is of the form "we can cover all requirements using at most k resources". The Koko Eating Bananas problem (LeetCode 875) is a canonical example: $\Phi(k)$:= "Koko can finish all piles within h hours at eating speed k".

4.5.4 Smallest-X-with-property

"Find the smallest X such that some derived quantity $f(X) \geq$ threshold". The Pie Progress problem and many geometry problems fall here.

4.6 When the Predicate is Itself a Search

A subtle but powerful technique arises when the predicate $\Phi(X)$ is itself a binary search. For example, in the problem "count the number of pairs (i, j) with $i < j$ and $a_i + a_j \leq X$", the predicate "for this X, are there at least k such pairs?" can be evaluated by sorting and using upper bound, giving $\mathcal{O}(n \log n)$ per evaluation. BSA over X then gives $\mathcal{O}(n \log n \log(\text{range}))$ overall, which beats the naive $\mathcal{O}(n^2)$ counting.

We will see this nested-search technique again in Chapter 7, where the predicate counts elements in an implicit multiset.

4.7 Chapter Summary

Binary search on the answer is the pattern of inverting a monotone feasibility predicate over an answer range. The four ingredients—a quantity to optimise, a feasibility predicate, a monotonicity proof, and a search range—together with the four-question procedure of §2.1 give a complete decision procedure for recognising BSA problems. The two worked examples (Aggressive Cows, Split Array Largest Sum) illustrate the pattern in its two most common forms, "maximise the minimum" and "minimise the maximum". The remaining chapters of Part II extend BSA to non-integer search spaces, implicit arrays, and parallel evaluation across multiple queries.

Chapter 5

Real-Valued Binary Search

When the answer is a real number rather than an integer, the binary search template of Chapter 1 requires two modifications: the loop termination condition changes (because "$\mathsf{hi} - \mathsf{lo} > 1$" no longer makes sense for reals), and the midpoint computation must use floating-point arithmetic with appropriate care for numerical stability. Real-valued binary search arises in geometry problems (smallest enclosing circle, line intersection parameters), in optimisation problems whose answer is a ratio or average, and in problems where the answer is the parameter of a continuous physical process.

5.1 Termination: Fixed Iteration Count versus EPS

There are two standard termination strategies for real-valued binary search.

EPS-based termination The loop runs while $\mathsf{hi} - \mathsf{lo} > \varepsilon$, for some small tolerance ε (typically 10^{-9}). The answer is taken as $(\mathsf{lo} + \mathsf{hi})/2$.

Fixed-iteration termination The loop runs a fixed number of iterations (typically 60 to 200), halving the interval each time. The answer is again $(\mathsf{lo} + \mathsf{hi})/2$.

Warning

EPS-based termination is fragile. If the search range is $[0, 10^9]$ and $\varepsilon = 10^{-9}$, the algorithm needs $\log_2(10^{18}) \approx 60$ iterations—but if the range is $[0, 10^{18}]$ with the same ε, it needs 120 iterations, and floating-point precision breaks down before the tolerance is reached. Worse, if the predicate evaluation involves large intermediate values, the loop may never converge to within ε at all due to accumulated rounding errors.

The fixed-iteration approach is almost always preferable. A double-precision float has 53 bits of mantissa, so 60 iterations of halving reduce the initial range by a factor of $2^{60} \approx 10^{18}$, exhausting the available precision. Running more than ~ 80 iterations is wasteful but harmless (the interval just stops shrinking); running fewer risks stopping prematurely.

5.2 Choosing the Iteration Count

A simple rule: run 60 iterations for double precision if the initial range is $[0, 1]$ or similar, 80 if the range spans many orders of magnitude, and 100 as a paranoid default. For `long double` (80-bit on x86, 64-bit mantissa), 80–100 iterations are appropriate. The constants are not critical; the extra iterations cost essentially nothing.

5.3 Reference Implementation

```cpp
// Returns the smallest x in [lo, hi] with p(x) == true.
// Precondition: p(hi) == true; p(lo) == false (open interval).
template <class Pred>
double first_true_real(double lo, double hi, Pred p, int iters = 80) {
    for (int i = 0; i < iters; i++) {
        double mid = (lo + hi) / 2.0;
        if (p(mid)) hi = mid;
        else        lo = mid;
    }
    return (lo + hi) / 2.0;
}
```

Listing 5.1: Real-valued binary search with fixed iteration count.

5.4 Worked Example: Computing the Cube Root

Problem 5.1 (Cube root)**.** Given a real number $x \in [0, 10^{18}]$, compute $\sqrt[3]{x}$ to within 10^{-6} absolute error.

Pattern identification. The function $f(t) = t^3$ is strictly increasing, so the predicate "is $t^3 \geq x$?" is monotone increasing in t. We are looking for the smallest t with $t^3 \geq x$, which is exactly $\sqrt[3]{x}$.

Approach. Binary search $t \in [0, 10^6]$ (since $(10^6)^3 = 10^{18}$) with the predicate $p(t) = (t^3 \geq x)$. Return the smallest t with $p(t)$ true.

Complexity. 80 iterations of $\mathcal{O}(1)$ each, plus the final rounding.

```cpp
double cube_root(double x) {
    double lo = 0.0, hi = 1e6 + 1.0;
    // invariant: lo^3 < x, hi^3 >= x
    for (int i = 0; i < 100; i++) {
        double mid = (lo + hi) / 2.0;
        if (mid * mid * mid >= x) hi = mid;
        else                      lo = mid;
    }
    return (lo + hi) / 2.0;
}
```

Listing 5.2: Cube root via real-valued binary search.

Tip

For *integer* cube roots, run a separate integer binary search and verify the result by cubing. Mixing integer and floating-point search in the same routine is a common source of bugs.

5.5 Worked Example: Maximum Average Subarray II

Problem 5.2 (LeetCode 644)**.** Given an array `nums` of n integers and an integer k, find the contiguous subarray of length at least k that has the maximum average value. Return this maximum average.

Pattern identification. The answer is real-valued (an average), and the tolerance (10^{-5}) signals real-valued BS. Q1 yes (maximising an average), Q2 yes (if an average $\geq \mu$ is achievable, so is every $\mu' < \mu$, by the same subarray), Q3 yes (we can check feasibility in $\mathcal{O}(n)$), Q4 yes (range is $[\min a_i, \max a_i]$).

Predicate.

$$\Phi(\mu) := \text{“there exists a subarray of length} \geq k \text{ with average} \geq \mu\text{”}.$$

Monotone decreasing in μ (we negate to get monotone increasing, or search for the largest μ with $\Phi(\mu)$ true).

Predicate evaluation. Subtract μ from every element; the question becomes “does there exist a subarray of length $\geq k$ with non-negative sum?”. This is checked in $\mathcal{O}(n)$ by maintaining the prefix sum and the minimum prefix sum at least k positions back.

Complexity. $\mathcal{O}(n \log(\text{range}/\varepsilon))$.

```cpp
class Solution {
public:
    double findMaxAverage(vector<int>& nums, int k) {
        double lo = *min_element(nums.begin(), nums.end());
        double hi = *max_element(nums.begin(), nums.end());
        // Search for largest mu with feasible(mu) == true.
        for (int it = 0; it < 100; it++) {
            double mid = (lo + hi) / 2.0;
            if (feasible(nums, k, mid)) lo = mid;
            else                        hi = mid;
        }
        return lo;
    }
private:
    // Is there a subarray of length >= k with average >= mu?
    bool feasible(const vector<int>& nums, int k, double mu) {
        int n = nums.size();
        vector<double> p(n + 1, 0.0);
        for (int i = 0; i < n; i++)
            p[i+1] = p[i] + (nums[i] - mu);
        double minP = 0.0;
        for (int i = k; i<=>n; i++) {
            minP = min(minP, p[i - k]);
            if (p[i] >= minP) return true;
        }
        return false;
    }
};
```

Listing 5.3: Solution to LeetCode 644.

5.6 Pitfalls Specific to Real-Valued Search

5.6.1 NaN propagation

If the predicate evaluation involves a division that can produce `NaN` (e.g., `0.0/0.0`), the comparison `mid >= x` will return `false` silently, and the search will diverge. Add explicit `std::isfinite` checks where division is involved.

5.6.2 Comparing with ε

If the problem requires comparing two reals within tolerance ε, use `fabs(a - b)< eps`, not `a == b`. This is basic floating-point hygiene but is often forgotten under time pressure.

5.6.3 Range too wide

If the initial range is $[0, 10^{18}]$ and the answer is around 10^{-6}, the search may exhaust precision before converging. In such cases, rescale the input so the answer lives in $[0, 1]$ or $[1, 10^6]$ before searching.

5.7 Chapter Summary

Real-valued binary search is integer binary search with two adjustments: termination is by fixed iteration count (typically 80), not by interval width; and the midpoint is computed as a true floating-point average. The two worked examples—cube root and maximum average subarray—illustrate the pattern in both pure numerical and array-feasibility settings. The pattern's main pitfalls are floating-point fragility, which the fixed-iteration convention largely avoids.

Chapter 6

Ternary Search on Unimodal Functions

Ternary search is the analogue of binary search for unimodal functions: functions that increase to a unique peak and then decrease (or vice versa). Where binary search exploits the *threshold* structure of a monotone predicate, ternary search exploits the *single-peak* structure of a unimodal objective. The algorithm maintains a candidate interval $[\mathsf{lo}, \mathsf{hi}]$ and, at each iteration, evaluates the function at two interior points, discarding the third of the interval that cannot contain the peak.

6.1 The Unimodal Property

Definition 6.1 (Unimodal function)**.** A function $f : S \to \mathbb{R}$ on a totally ordered set S is *unimodal* (with a maximum) if there exists $x^* \in S$ such that f is strictly increasing on $\{x \in S : x \le x^*\}$ and strictly decreasing on $\{x \in S : x \ge x^*\}$. Unimodality with a minimum is defined dually.

Theorem 6.2 (Ternary search)**.** *Let f be a unimodal function on $[L, R]$. Define*

$$m_1 = \mathsf{lo} + \frac{\mathsf{hi} - \mathsf{lo}}{3}, \qquad m_2 = \mathsf{hi} - \frac{\mathsf{hi} - \mathsf{lo}}{3}.$$

If $f(m_1) < f(m_2)$, the maximum lies in $[m_1, \mathsf{hi}]$; otherwise it lies in $[\mathsf{lo}, m_2]$. Iterating this reduces the interval by a factor of $2/3$ per step, so $\log_{3/2}((R - L)/\varepsilon)$ iterations suffice for ε accuracy.

Proof. By unimodality, f is strictly increasing on $[\mathsf{lo}, x^*]$ and strictly decreasing on $[x^*, \mathsf{hi}]$. Suppose $f(m_1) < f(m_2)$. If $x^* \le m_1$, then f is decreasing on $[m_1, \mathsf{hi}]$, so $f(m_1) > f(m_2)$, contradicting the assumption. Hence $x^* > m_1$ and the maximum lies in $[m_1, \mathsf{hi}]$. The dual case is symmetric. □

6.2 Discrete versus Continuous Ternary Search

Ternary search applies to both discrete and continuous search spaces. The discrete version is useful when the search space is a small integer range (e.g., $[0, n-1]$) and the function evaluation is exact; the continuous version applies to real-valued search spaces with floating-point function evaluation.

For the discrete case, the loop terminates when $\mathsf{hi} - \mathsf{lo} \le 2$ and the maximum is found by linear scan over the surviving 2–3 candidates. For the continuous case, the loop runs a fixed number of iterations (typically 200 for double precision), as in real-valued binary search.

6.3 Reference Implementation

```cpp
// Discrete ternary search: maximise f over integer range [lo, hi].
// Precondition: f is unimodal on [lo, hi].
template <class F>
long long argmax_ternary(long long lo, long long hi, F f) {
    while (hi - lo > 2) {
        long long m1 = lo + (hi - lo) / 3;
        long long m2 = hi - (hi - lo) / 3;
        if (f(m1) < f(m2)) lo = m1;
        else               hi = m2;
    }
    long long best = lo;
    for (long long i = lo; i<=>hi; i++)
        if (f(i) > f(best)) best = i;
    return best;
}

// Continuous ternary search: maximise f over [lo, hi].
template <class F>
double argmax_ternary_real(double lo, double hi, F f, int iters = 200) {
    for (int i = 0; i < iters; i++) {
        double m1 = lo + (hi - lo) / 3.0;
        double m2 = hi - (hi - lo) / 3.0;
        if (f(m1) < f(m2)) lo = m1;
        else               hi = m2;
    }
    return (lo + hi) / 2.0;
}
```

Listing 6.1: Ternary search for the maximum of a unimodal function.

6.4 Worked Example: Minimising the Sum of Distances

Problem 6.3 (Facility location on a line)**.** Given n points $x_1 < x_2 < \cdots < x_n$ on a line, find the point y that minimises $\sum_i |x_i - y|$.

Pattern identification. The function $f(y) = \sum_i |x_i - y|$ is convex (a sum of convex functions is convex), hence unimodal with a minimum. Ternary search applies.

Approach. Ternary-search $y \in [x_1, x_n]$ minimising $f(y)$. Each evaluation of f costs $\mathcal{O}(n)$.

Complexity. $\mathcal{O}(n \log(\text{range}/\varepsilon))$.

Remark. For this specific problem, the exact answer is the *median* of the x_i, which can be found in $\mathcal{O}(n)$ by `nth_element`. Ternary search is overkill here but illustrates the pattern; it is the right tool when the function is convex but has no closed-form solution.

6.5 When Ternary Search Fails

Ternary search requires *strict* unimodality. Two failure modes are common.

Plateaus If f is constant on an interval, ternary search may oscillate, because the comparison $f(m_1) < f(m_2)$ is false and the comparison $f(m_1) > f(m_2)$ is also false. The fix is to require *strict* inequality, with $f(m_1) \le f(m_2)$ treated as "the peak is in the right half".

Non-unimodal functions If f has multiple local maxima, ternary search converges to one of them, but not necessarily the global one. There is no general fix; the problem must be reformulated to expose a unimodal structure, or a different technique (e.g., simulated annealing, branch-and-bound) must be used.

Warning

A common pitfall is to attempt ternary search on a function that is *discrete-convex but not strictly unimodal*. For example, $f(i) = \binom{n}{i}$ is unimodal with a maximum at $i = n/2$, but for even n it takes the same value at $i = n/2 - 1$ and $i = n/2$. The standard ternary search handles this correctly only if the comparison is *not* strict. Verify by testing the predicate on a constant function before submitting.

6.6 Golden-Section Search

A variant of ternary search called *golden-section search* reduces the per-iteration cost from two function evaluations to one (by reusing one of the previous probes). The constant factor improvement is $2\times$ for the discrete case and may matter when f is expensive. The convergence rate drops from $2/3$ per iteration to $\varphi^{-1} \approx 0.618$ per iteration, where $\varphi = (1 + \sqrt{5})/2$ is the golden ratio. The asymptotic iteration count is therefore $\log_\varphi((R - L)/\varepsilon)$, which is about $1.4\times$ the ternary count; the constant factor saves only one evaluation per iteration, so golden-section is faster by a small constant. For most CP applications, plain ternary search is sufficient.

6.7 Chapter Summary

Ternary search inverts the unimodality property of a function to locate its peak in $\mathcal{O}\left(\log_{3/2}\right)$ evaluations. It is the right tool whenever the objective is convex or concave and no closed-form solution is available. The two main failure modes—plateaus and non-unimodality—are guarded against by careful predicate design and by the strict-inequality convention. For most practical purposes, plain ternary search with a fixed iteration count is preferable to the more elaborate golden-section variant.

Chapter 7

Binary Search on Implicit Arrays

The implicit array pattern applies when the problem asks for a "k-th smallest/largest" element of a multiset that is too large to materialise but whose elements can be *counted* below a given threshold. The trick is to binary search the value of the answer, with the predicate "are there at least k elements $\leq x$?". The predicate is evaluated by an efficient counting procedure that never enumerates the elements.

7.1 The Pattern

Let M be a multiset of $|M| = N$ elements, with N potentially huge (e.g., $N = n^2$ for the multiset of pairwise sums of an n-element array). We want the k-th smallest element of M. Direct materialisation costs $\mathcal{O}(N)$, which is too much. But suppose we have a function

$$\textsc{CountBelow}(x) := |\{m \in M : m \leq x\}|$$

that runs in time $T(n)$ (typically $\mathcal{O}(n)$ or $\mathcal{O}(n \log n)$). Then the k-th smallest element is the smallest x with $\textsc{CountBelow}(x) \geq k$, found by binary search over the value range in $\mathcal{O}(T(n) \log(\text{range}))$.

Key Idea

Implicit-array recipe.

1. Identify the multiset M whose k-th element you seek.
2. Design a $\mathcal{O}(n)$ (or $\mathcal{O}(n \log n)$) procedure to compute $\textsc{CountBelow}(x)$ for a given x.
3. Binary search the smallest x with $\textsc{CountBelow}(x) \geq k$.

7.2 Worked Example: K-th Smallest in a Sorted Matrix

Problem 7.1 (LeetCode 378)**.** Given an $n \times n$ matrix where each row and each column is sorted in ascending order, return the k-th smallest element in the matrix.

Pattern identification. The matrix has n^2 elements; for $n = 100$ this is 10^4, manageable, but for $n = 10^5$ it is 10^{10}, not. The "k-th smallest" cue signals implicit-array BS.

Counting procedure. For a given x, walk from the top-right corner of the matrix: start at $(0, n-1)$, and at each step, if $a[i][j] \leq x$, add $j+1$ to the count (all elements in row i up to column j are $\leq x$, since the row is sorted) and move down to $(i+1, j)$; otherwise move left to $(i, j-1)$. This visits at most $2n$ cells, giving $\mathcal{O}(n)$ per predicate evaluation.

Search range. $[a[0][0], a[n-1][n-1]]$.
Complexity. $\mathcal{O}(n \log(\text{range}))$.

```cpp
class Solution {
public:
    int kthSmallest(vector<vector<int>>& mat, int k) {
        int n = mat.size();
        int lo = mat[0][0], hi = mat[n-1][n-1];
        while (lo < hi) {
            int mid = lo + (hi - lo) / 2;
            int cnt = count_le(mat, mid);
            if (cnt >= k) hi = mid;
            else          lo = mid + 1;
        }
        return lo;
    }
private:
    int count_le(const vector<vector<int>>& mat, int x) {
        int n = mat.size();
        int cnt = 0, j = n - 1;
        for (int i = 0; i < n; i++) {
            while (j >= 0 && mat[i][j] > x) j--;
            cnt += (j + 1);
        }
        return cnt;
    }
};
```

Listing 7.1: Solution to LeetCode 378.

Remark. Note the convention here is closed-interval (`lo < hi` loop, `lo = mid + 1` update) rather than open-interval. This is because the predicate is "$\geq k$" and we want the smallest x where it holds; the closed-interval convention is natural for "smallest with property". Either convention works; pick the one that requires fewer special cases.

7.3 Worked Example: Median of Two Sorted Arrays

Problem 7.2 (LeetCode 4)**.** Given two sorted arrays `nums1` and `nums2` of sizes m and n, return the median of the combined sorted array in $\mathcal{O}(\log(m+n))$ time.

Pattern identification. The combined array has $m+n$ elements; materialising it costs $\mathcal{O}(m+n)$, which violates the required $\mathcal{O}(\log)$ bound. The "k-th smallest" cue combined with the explicit $\mathcal{O}(\log)$ bound signals implicit-array BS.

Approach. The median is the $\lceil (m+n)/2 \rceil$-th smallest element. To find the k-th smallest of two sorted arrays A and B, compare $A[k/2-1]$ and $B[k/2-1]$ (with sentinel handling for out-of-range indices); the smaller one's first $k/2$ elements cannot contain the k-th smallest, so discard them and recurse on the remaining arrays with k reduced by $k/2$.

Complexity. Each recursive call halves k, so $\mathcal{O}(\log k) = \mathcal{O}(\log(m+n))$ total.

```cpp
class Solution {
public:
    double findMedianSortedArrays(vector<int>& A, vector<int>& B) {
        int m = A.size(), n = B.size();
        int total = m + n;
        int k1 = (total + 1) / 2;
        int k2 = (total + 2) / 2;
        return (kth(A, 0, B, 0, k1) + kth(A, 0, B, 0, k2)) / 2.0;
    }
private:
```

```
    // k-th smallest (1-indexed) of A[a..] and B[b..].
    int kth(const vector<int>& A, int a, const vector<int>& B, int b, int k) {
        int m = A.size() - a, n = B.size() - b;
        if (m < n) return kth(B, b, A, a, k);  // ensure A is the longer
        if (n == 0) return A[a + k - 1];
        if (k == 1) return min(A[a], B[b]);
        int i = min(m, k / 2), j = min(n, k / 2);
        if (A[a + i - 1] < B[b + j - 1])
            return kth(A, a + i, B, b, k - i);
        else
            return kth(A, a, B, b + j, k - j);
    }
};
```

Listing 7.2: Solution to LeetCode 4: k-th smallest of two sorted arrays.

7.4 Worked Example: K-th Smallest Pair Distance

Problem 7.3 (LeetCode 719)**.** Given an integer array `nums`, return the k-th smallest *absolute distance* among all pairs. The pair distance is $|a_i - a_j|$ for $i < j$.

Pattern identification. The multiset of pair distances has $\binom{n}{2}$ elements—up to $\sim 5 \times 10^9$ for $n = 10^5$, far too many to materialise. The "k-th smallest" cue signals implicit-array BS.

Counting procedure. Sort the array. For a given distance d, the count of pairs with $|a_i - a_j| \le d$ is computed by two pointers: for each i from left to right, advance a pointer $j \ge i$ as long as $a_j - a_i \le d$; add $(j - i)$ to the count. This is $\mathcal{O}(n)$.

Search range. $[0, \max a - \min a]$.

Complexity. $\mathcal{O}(n \log n + n \log(\text{range}))$.

```
class Solution {
public:
    int smallestDistancePair(vector<int>& nums, int k) {
        sort(nums.begin(), nums.end());
        int lo = 0, hi = nums.back() - nums.front();
        while (lo < hi) {
            int mid = lo + (hi - lo) / 2;
            if (count_le(nums, mid) >= k) hi = mid;
            else                          lo = mid + 1;
        }
        return lo;
    }
private:
    long long count_le(const vector<int>& a, int d) {
        long long cnt = 0;
        int j = 0;
        for (int i = 0; i < (int)a.size(); i++) {
            while (j < (int)a.size() && a[j] - a[i]<=>d) j++;
            cnt += (j - i - 1);
        }
        return cnt;
    }
};
```

Listing 7.3: Solution to LeetCode 719.

7.5 A Catalogue of Counting Procedures

The art of implicit-array BS is the design of the counting procedure. We catalogue the most common.

Sorted matrix Walk top-right to bottom-left; cost $\mathcal{O}(m+n)$.

Two sorted arrays Two pointers; cost $\mathcal{O}(m+n)$, or $\mathcal{O}(\log)$ via the recursive median-of-medians trick in §7.3.

Pairwise differences/s sums of sorted array Two pointers after sorting; cost $\mathcal{O}(n)$.

Subsets of bounded size Iterate over subsets of half size, sort one half, binary search the other; cost $\mathcal{O}\left(2^{n/2}\log 2^{n/2}\right) = \mathcal{O}\left(n2^{n/2}\right)$ (meet-in-the-middle).

Geometric configurations Sweep line with appropriate data structure; cost depends on geometry.

7.6 Chapter Summary

The implicit-array pattern inverts a counting predicate over a large multiset to locate its k-th element in $\mathcal{O}(T(n)\log(\text{range}))$, where $T(n)$ is the per-query counting cost. The three worked examples—sorted matrix, two sorted arrays, pair distances—illustrate the pattern in three different counting regimes. The pattern's key skill is designing the counting procedure, which is problem-specific and rewards practice.

Chapter 8

Parallel Binary Search

Parallel binary search—hereafter *PBS*—is the technique of running many binary searches simultaneously, sharing the work of predicate evaluation across all of them. It applies when the problem describes many queries, each asking "what is the smallest X such that $\Phi_i(X)$ holds?", and the predicate $\Phi_i(X)$ involves a data structure that can be advanced *in time*. Instead of binary-searching each query independently and re-evaluating the predicate from scratch each time, PBS runs $\mathcal{O}(\log T)$ rounds; in each round, all queries' mid-points are grouped together and the data structure is advanced *once* across the round's midpoint.

8.1 When PBS Applies

The structural cues for PBS (cf. §2.5) are:

- Many queries (Q of them, with Q up to $\sim 10^5$), each asking "when does event E_i first occur?".
- A sequence of T *temporal events* (e.g., edge additions to a graph, array updates) such that $\Phi_i(X)$ can be rephrased as "after the first X events, is query i satisfied?".
- The data structure needed to evaluate Φ_i after X events can be *advanced* from $X-1$ to X events incrementally, but *cannot* be cheaply queried for an arbitrary X out of order.

The third cue is essential. If predicates could be evaluated out of order, we would binary-search each query independently with no sharing. PBS is the right tool when the data structure is *incremental*—a union-find forest with edge additions, a segment tree with point updates, a sweep-line state—and the cost of *rebuilding* it for each query's binary search would dominate.

8.2 The Algorithm

Algorithm 2 Parallel binary search

Initialise: for each query i, set $\mathsf{lo}_i \leftarrow 0$, $\mathsf{hi}_i \leftarrow T+1$.
while some query has $\mathsf{hi}_i - \mathsf{lo}_i > 1$ **do**
  Group queries by their midpoint $m_i = \lfloor (\mathsf{lo}_i + \mathsf{hi}_i)/2 \rfloor$.
  Reset the data structure to its state at time 0.
  for each time t from 1 to T, in order **do**
    Apply event t to the data structure.
    for each query i with $m_i = t$ **do**
      if Φ_i holds in the current data-structure state **then**
        $\mathsf{hi}_i \leftarrow t$
      else
        $\mathsf{lo}_i \leftarrow t$
      end if
    end for
  end for
end while
return $(\mathsf{hi}_i)_{i=1}^{Q}$

The key observation is that the inner loop over t processes the events *once per round*, applying each event exactly once. Queries are dispatched to the appropriate t by their midpoint, so the cost of advancing the data structure through T events is paid $\mathcal{O}(\log T)$ times total—once per round—rather than $Q \log T$ times as in naive per-query binary search.

Theorem 8.1 (Complexity of PBS)**.** *Suppose each event update costs $\mathcal{O}(U)$, each predicate check costs $\mathcal{O}(C)$, and there are Q queries and T events. Then PBS terminates in $\mathcal{O}(\log T)$ rounds, with total cost $\mathcal{O}((T \cdot U + Q \cdot C) \log T)$.*

Compare this with naive per-query binary search, which would cost $\mathcal{O}(Q \cdot T \cdot U \log T)$ (each query re-applies T events per predicate evaluation). When $Q \approx T$, PBS is $\mathcal{O}(T)$ times faster.

8.3 Worked Example: Dynamic Connectivity Offline

Problem 8.2 (Offline dynamic connectivity, simplified)**.** You are given a graph with n vertices and a sequence of m edge-additions (no deletions). You are also given q queries of the form "are vertices u_i and v_i connected after the first X_i edge additions?". Answer all queries.

Pattern identification. The structural cues for PBS are all present: many queries (q of them), each asking about the state of a dynamic data structure (connectivity) at a specific time X_i. The data structure (union-find) is incremental but cannot be queried for an arbitrary X out of order without rebuilding.

Approach. Run PBS. In each round, reset the union-find, apply edges in order, and at each query's midpoint check whether u_i and v_i are in the same component. The answer for query i is the smallest X at which they become connected.

Complexity. $\mathcal{O}((m \cdot \alpha(n) + q \cdot \alpha(n)) \log m) = \mathcal{O}((m+q)\alpha(n) \log m)$, where α is the inverse Ackermann function from union-find.

```
struct DSU {
    vector<int> p, r;
    DSU(int n) : p(n), r(n, 0) { iota(p.begin(), p.end(), 0); }
    int find(int x) { return p[x] == x ? x : p[x] = find(p[x]); }
```

```
    bool unite(int a, int b) {
        a = find(a); b = find(b);
        if (a == b) return false;
        if (r[a] < r[b]) swap(a, b);
        p[b] = a;
        if (r[a] == r[b]) r[a]++;
        return true;
    }
    bool connected(int a, int b) { return find(a) == find(b); }
};

// edges: sequence of edge additions (u, v)
// queries: each is (u, v, X) asking "are u, v connected after first X edges?"
vector<int> pbs_connectivity(
    int n,
    const vector<pair<int,int>>& edges,
    vector<tuple<int,int,int>>& queries)  // (u, v, X)
{
    int m = edges.size();
    int q = queries.size();
    vector<int> lo(q, 0), hi(q, m + 1);
    // hi[i] = m+1 means "never connected within m edges"
    vector<int> ans(q, m + 1);

    while (true) {
        // Group queries by midpoint.
        vector<vector<int>> at_mid(m + 2);
        bool any = false;
        for (int i = 0; i < q; i++) {
            if (hi[i] - lo[i] > 1) {
                any = true;
                int mid = (lo[i] + hi[i]) / 2;
                at_mid[mid].push_back(i);
            }
        }
        if (!any) break;

        DSU dsu(n);
        for (int t = 1; t<=>m; t++) {
            dsu.unite(edges[t-1].first, edges[t-1].second);
            for (int i : at_mid[t]) {
                auto [u, v, X] = queries[i];
                if (dsu.connected(u, v)) {
                    hi[i] = t;
                    ans[i] = t;
                } else {
                    lo[i] = t;
                }
            }
        }
        // Queries with mid = m+1: their hi = m+1 already, check connectivity
            at m
        for (int i : at_mid[m + 1]) {
            auto [u, v, X] = queries[i];
            if (dsu.connected(u, v)) {
                hi[i] = m;
                ans[i] = m;
            } else {
                lo[i] = m;
            }
        }
    }
    return ans;  // ans[i] = smallest #edges after which u_i, v_i are connected
```

```
67 }
```

Listing 8.1: Parallel binary search for offline dynamic connectivity.

8.4 Worked Example: Meteors (SPOJ)

Problem 8.3 (SPOJ Meteors)**.** There are N monitoring stations and M sectors. Each sector belongs to one station. A sequence of K meteor showers occurs; shower i adds meteors to a range $[l_i, r_i]$ of sectors (circular) by amount a_i. Each station has a required total meteor count r_j. For each station, find the earliest shower after which its requirement is met.

Pattern identification. Each station's question is "smallest X such that my total $\geq r_j$"—a BSA predicate. The events are the showers, and the data structure is a range-add point-query structure (Fenwick tree with range updates). The predicate is monotone (totals only grow), and the data structure is incremental. PBS applies.

Approach. PBS with a Fenwick tree supporting range add and point query. In each round, apply the showers in order, and at each query's midpoint check whether the station's total has reached r_j.

Complexity. $\mathcal{O}((K \log M + N \log M) \log K) = \mathcal{O}((K + N) \log M \log K)$.

We omit the full implementation; the structure mirrors Listing 8.1 with the DSU replaced by a Fenwick tree.

8.5 PBS with Rollback

A subtle variant arises when the data structure supports *rollback*—undoing the most recent update. In this case, the inner loop of PBS can be reorganised: instead of resetting the data structure at the start of each round and re-applying all events, we keep the data structure persistent across rounds and *roll back* to the appropriate point. This reduces the cost from $\mathcal{O}(T \log T)$ event applications to $\mathcal{O}(T)$ (each event is applied and rolled back at most once per round, but the total work is amortised).

The rollback variant is essential when event updates are expensive (e.g., $\mathcal{O}\left(\log^2 n\right)$ each) and T is large. The implementation requires the data structure to be *rollback-able*, which usually means avoiding path compression in union-find and using only undo-able operations (e.g., simple stack-based segment tree updates).

8.6 Chapter Summary

Parallel binary search is the right tool for offline problems with many monotone queries over a sequence of events. It amortises the cost of advancing the data structure across $\mathcal{O}(\log T)$ rounds, giving a total cost of $\mathcal{O}((T \cdot U + Q \cdot C) \log T)$ versus the naive $\mathcal{O}(Q \cdot T \cdot U \log T)$. The pattern's identifying cue is the presence of many queries, each asking about the state of an incremental data structure at a specific time.

Chapter 9

Binary Lifting

Binary lifting is the technique of precomputing the results of *iterated applications* of a function at power-of-two counts, then composing these precomputed results to answer "apply f k times" queries in $\mathcal{O}(\log k)$. The paradigmigmatic application is the lowest common ancestor (LCA) in a rooted tree, but the technique applies more broadly: k-th ancestor, functional graph cycle detection, and certain dynamic programming transitions.

9.1 The Pattern

Let $f : S \to S$ be a function on a finite set S. We want to answer queries of the form "what is $f^{(k)}(x)$?" for arbitrary $x \in S$ and $k \in \mathbb{N}$. The naive approach costs $\mathcal{O}(k)$ per query. Binary lifting precomputes a table

$$\mathrm{up}[i][v] := f^{(2^i)}(v)$$

for $i = 0, 1, \ldots, \lceil \log_2 K_{\max} \rceil$ and all $v \in S$. The table has size $\mathcal{O}(|S| \log K_{\max})$ and is filled in $\mathcal{O}(|S| \log K_{\max})$ time by the recurrence

$$\mathrm{up}[i][v] = \mathrm{up}[i-1][\mathrm{up}[i-1][v]].$$

A query $f^{(k)}(x)$ is answered in $\mathcal{O}(\log k)$ by decomposing k in binary and applying the corresponding $\mathrm{up}[i]$ entries.

9.2 Reference Implementation

```
struct BinaryLifting {
    int n, LOG;
    vector<vector<int>> up;  // up[i][v] = f^(2^i)(v)
    BinaryLifting(int n_, const vector<int>& f, long long Kmax)
        : n(n_), LOG(0) {
        while ((1LL << LOG) <= Kmax) LOG++;
        up.assign(LOG, vector<int>(n));
        up[0] = f;
        for (int i = 1; i < LOG; i++)
            for (int v = 0; v < n; v++)
                up[i][v] = up[i-1][up[i-1][v]];
    }
    // Apply f to x exactly k times.
    int apply(int x, long long k) const {
        for (int i = 0; i < LOG && k > 0; i++) {
            if (k & 1) x = up[i][x];
            k >>= 1;
        }
```

```
        return x;
    }
};
```

Listing 9.1: Generic binary lifting for a function $f : S \to S$ over $\{0, \ldots, n-1\}$.

9.3 Worked Example: LCA on a Tree

Problem 9.1 (LCA)**.** Given a rooted tree with n vertices, answer queries of the form "what is the lowest common ancestor of u and v?".

Pattern identification. LCA can be reduced to "k-th ancestor": bring u and v to the same depth, then lift both simultaneously until they meet. The k-th ancestor query is a classic binary lifting application.

Approach. Preprocess:

1. Compute depth $d[v]$ for each vertex via DFS.
2. Set $f(v) = \text{parent}(v)$ (with parent(root) = root).
3. Build the binary lifting table on f.

Query (u, v): bring the deeper of u, v up to the depth of the shallower; then lift both in lockstep by the largest power of two that keeps them distinct. When no such lift is possible, the parent of either vertex is the LCA.

Complexity. $\mathcal{O}(n \log n)$ preprocessing, $\mathcal{O}(\log n)$ per query.

```
struct LCA {
    int n, LOG;
    vector<vector<int>> up;
    vector<int> depth;
    const vector<vector<int>>& adj;

    LCA(int root, const vector<vector<int>>& g)
        : n(g.size()), LOG(0), adj(g) {
        while ((1 << LOG) < n) LOG++;
        up.assign(LOG, vector<int>(n));
        depth.assign(n, 0);
        up[0][root] = root;
        dfs(root, root);
        for (int i = 1; i < LOG; i++)
            for (int v = 0; v < n; v++)
                up[i][v] = up[i-1][up[i-1][v]];
    }
    void dfs(int v, int p) {
        up[0][v] = p;
        for (int u : adj[v]) if (u != p) {
            depth[u] = depth[v] + 1;
            dfs(u, v);
        }
    }
    int kth_ancestor(int v, int k) const {
        for (int i = 0; i < LOG && k > 0; i++) {
            if (k & 1) v = up[i][v];
            k >>= 1;
        }
        return v;
    }
    int lca(int u, int v) const {
        if (depth[u] < depth[v]) swap(u, v);
        u = kth_ancestor(u, depth[u] - depth[v]);
```

```
        if (u == v) return u;
        for (int i = LOG - 1; i >= 0; i--) {
            if (up[i][u] != up[i][v]) {
                u = up[i][u];
                v = up[i][v];
            }
        }
        return up[0][u];
    }
    int dist(int u, int v) const {
        return depth[u] + depth[v] - 2 * depth[lca(u, v)];
    }
};
```

Listing 9.2: LCA via binary lifting.

9.4 Worked Example: Functional Graph Cycle Detection

Problem 9.2 (Functional graph)**.** Given a functional graph G (each vertex has out-degree 1) on n vertices, answer queries of the form "starting at vertex v, what is the k-th vertex visited?" ($k \le 10^{18}$).

Pattern identification. The graph is functional, so "k-th vertex visited from v" is exactly $f^{(k)}(v)$ where f is the successor function. The huge k (10^{18}) rules out simulation; binary lifting is the canonical tool.

Approach. Build the lifting table on f and answer each query by decomposing k in binary.

Complexity. $\mathcal{O}(n \log K_{\max})$ preprocessing, $\mathcal{O}(\log K_{\max})$ per query.

9.5 Variations

9.5.1 Sparse tables for RMQ

The sparse table for range-minimum queries is a binary lifting application: $\mathrm{st}[i][j]$ stores the minimum of the range $[j, j + 2^i)$, and the recurrence

$$\mathrm{st}[i][j] = \min(\mathrm{st}[i-1][j], \mathrm{st}[i-1][j + 2^{i-1}])$$

builds the table in $\mathcal{O}(n \log n)$. Range queries are answered in $\mathcal{O}(1)$ by overlapping two power-of-two ranges.

9.5.2 Heavy-light decomposition

Binary lifting is the simplest LCA algorithm but not the fastest; *heavy-light decomposition* gives $\mathcal{O}\left(\log^2 n\right)$ queries with a smaller constant in some cases, and Tarjan's offline algorithm gives $\mathcal{O}(n + q\alpha(n))$ total. Binary lifting remains the easiest to implement correctly under time pressure.

9.6 Chapter Summary

Binary lifting precomputes $f^{(2^i)}(v)$ for power-of-two i and composes these to answer iterated-function queries in $\mathcal{O}(\log k)$. The technique applies to LCA, k-th ancestor, functional graph queries, and (via sparse tables) range minimum queries. The preprocessing cost is $\mathcal{O}(n \log K_{\max})$ in space and time, which is acceptable for $n \le 10^5$ and $K_{\max} \le 10^{18}$.

Chapter 10

Divide and Conquer DP Optimisation

Divide-and-conquer DP optimisation—hereafter *D&C DP*—is a technique for accelerating certain 1D/1D dynamic programs whose cost function satisfies the *quadrangle inequality*. The naive DP costs $\mathcal{O}(n^2)$; D&C DP reduces this to $\mathcal{O}(n \log n)$ by exploiting a monotonicity property of the optimal decision pointer.

10.1 The Setting

A *1D/1D DP* has the form

$$dp[i] = \min_{j<i} \{dp[j] + w(j, i)\},$$

where $w : \{0, \dots, n-1\}^2 \to \mathbb{R}$ is a cost function and the base case $dp[0]$ is given. The naive DP evaluates each $dp[i]$ in $\mathcal{O}(i)$ by trying all $j < i$, giving total cost $\mathcal{O}(n^2)$.

10.2 The Quadrangle Inequality

Definition 10.1 (Quadrangle inequality)**.** The cost function w satisfies the *quadrangle inequality* (QI) if for all $a \le b \le c \le d$,

$$w(a, c) + w(b, d) \le w(a, d) + w(b, c).$$

Equivalently, w satisfies the *inverse QI* if the inequality is reversed; we focus on QI for minimisation.

Definition 10.2 (Optimal decision pointer)**.** Let $\mathrm{opt}(i) := \arg\min_{j<i} \{dp[j] + w(j, i)\}$, breaking ties towards the smaller j.

Theorem 10.3 (Monotonicity of the optimal decision pointer)**.** *If w satisfies the quadrangle inequality, then $opt(1) \le opt(2) \le \cdots \le opt(n-1)$.*

Proof sketch. Suppose for contradiction that $\mathrm{opt}(i) > \mathrm{opt}(i+1)$ for some i. Set $a = \mathrm{opt}(i+1)$, $b = \mathrm{opt}(i)$, $c = i$, $d = i+1$. By optimality,

$$dp[a] + w(a, i+1) \le dp[b] + w(b, i+1),$$
$$dp[b] + w(b, i) \le dp[a] + w(a, i).$$

Adding these and cancelling $dp[a] + dp[b]$:

$$w(a, i+1) + w(b, i) \le w(b, i+1) + w(a, i).$$

But by QI applied to $a \le b \le i \le i+1$,

$$w(a,i) + w(b,i+1) \le w(a,i+1) + w(b,i),$$

which contradicts the previous inequality unless both are equalities. Equality implies $\text{opt}(i+1) = \text{opt}(i)$ by tie-breaking, contradicting the assumption. □

10.3 The D&C DP Algorithm

The monotonicity of opt allows us to compute dp in $\mathcal{O}(n \log n)$ by divide and conquer:

Algorithm 3 D&C DP optimisation

```
1: function Solve(l, r, optL, optR)        ▷ Computes dp[i] for i ∈ [l, r], knowing opt(i) ∈ [optL, optR].
2:     if l > r then return
3:     end if
4:     m ← ⌊(l + r)/2⌋
5:     opt_m ← optL
6:     dp[m] ← dp[optL] + w(optL, m)
7:     for j = optL + 1 to min(optR, m − 1) do
8:         if dp[j] + w(j, m) < dp[m] then
9:             dp[m] ← dp[j] + w(j, m)
10:            opt_m ← j
11:        end if
12:    end for
13:    Solve(l, m − 1, optL, opt_m)
14:    Solve(m + 1, r, opt_m, optR)
15: end function
```

The recurrence is $T(n) = 2T(n/2) + \mathcal{O}(n)$, giving $T(n) = \mathcal{O}(n \log n)$.

10.4 Worked Example: Post Office Problem

Problem 10.4 (IOI 2000 Post Office)**.** There are V villages on a line at positions $x_1 < x_2 < \cdots < x_V$, and P post offices to build. A post office built at village i serves a contiguous range of villages; the cost of serving villages $[j, i]$ from a post office at village $m \in [j, i]$ is $\sum_{k=j}^{i} |x_k - x_m|$. Find the minimum total cost to serve all villages using P post offices.

Pattern identification. The DP is

$$dp[p][i] = \min_{j<i} \{dp[p-1][j] + w(j+1, i)\},$$

where $w(j+1, i)$ is the cost of serving villages $[j+1, i]$ from a single optimally-placed post office. The cost function w is the sum of distances to the median of the range, which satisfies the quadrangle inequality (we verify this informally: the cost is convex in the range endpoints, which is sufficient). D&C DP applies, reducing the per-layer cost from $\mathcal{O}(V^2)$ to $\mathcal{O}(V \log V)$.

Complexity. $\mathcal{O}(P \cdot V \log V)$ for the DP layers, plus $\mathcal{O}(V^2)$ precomputation of w (which can be reduced to $\mathcal{O}(V^2)$ via prefix sums).

```
#include <bits/stdc++.h>
using namespace std;
typedef long long ll;
```

```
4
5  int V, P;
6  vector<ll> x;        // village positions
7  vector<vector<ll>> w;  // w[l][r] = cost of serving [l, r] from one office
8  vector<vector<ll>> dp; // dp[p][i] = min cost to serve [1..i] with p offices
9  vector<int> opt;
10
11 void compute_w() {
12     w.assign(V + 1, vector<ll>(V + 1, 0));
13     for (int l = 1; l<=>V; l++) {
14         int m = l;  // median index, slides right as r grows
15         ll leftSum = 0, rightSum = 0, leftCnt = 0;
16         // ... (omitted for brevity; standard prefix-sum trick)
17         // For each r >= l, w[l][r] = w[l][r-1] + (x[r] - x[m])
18         // with m updated as r grows.
19     }
20     // Simplified: O(V^2) using precomputed prefix sums of positions.
21     vector<ll> pref(V + 1, 0);
22     for (int i = 1; i<=>V; i++) pref[i] = pref[i-1] + x[i];
23     for (int l = 1; l<=>V; l++)
24         for (int r = l; r<=>V; r++) {
25             int m = (l + r) / 2;
26             ll leftCost  = x[m] * (m - l + 1) - (pref[m] - pref[l-1]);
27             ll rightCost = (pref[r] - pref[m]) - x[m] * (r - m);
28             w[l][r] = leftCost + rightCost;
29         }
30 }
31
32 void solve_layer(int p, int l, int r, int optL, int optR) {
33     if (l > r) return;
34     int m = (l + r) / 2;
35     dp[p][m] = LLONG_MAX;
36     int best = optL;
37     int hi = min(optR, m - 1);
38     for (int j = optL; j<=>hi; j++) {
39         ll val = dp[p-1][j] + w[j+1][m];
40         if (val < dp[p][m]) {
41             dp[p][m] = val;
42             best = j;
43         }
44     }
45     solve_layer(p, l, m - 1, optL, best);
46     solve_layer(p, m + 1, r, best, optR);
47 }
48
49 int main() {
50     cin >> V >> P;
51     x.assign(V + 1, 0);
52     for (int i = 1; i<=>V; i++) cin >> x[i];
53     compute_w();
54     dp.assign(P + 1, vector<ll>(V + 1, LLONG_MAX));
55     dp[0][0] = 0;
56     for (int p = 1; p<=>P; p++)
57         solve_layer(p, 1, V, 0, V - 1);
58     cout << dp[P][V] << "\n";
59     return 0;
60 }
```

Listing 10.1: Solution to the Post Office problem using D&C DP.

10.5 Variations and Related Optimisations

10.5.1 Knuth optimisation

When the cost function w satisfies both the quadrangle inequality and the *monotonicity condition* $w(b,c) \leq w(a,d)$ for $a \leq b \leq c \leq d$, the optimal decision pointer satisfies $\text{opt}(i-1) \leq \text{opt}(i) \leq \text{opt}(i+1)$. This stronger monotonicity allows a $\mathcal{O}(n^2/\log n)$ optimisation via Knuth's technique, which is asymptotically worse than D&C DP but has a smaller constant.

10.5.2 Aliens (Lagrangian relaxation)

When the DP has a *cardinality constraint* ("use at most k transitions"), the Lagrangian relaxation (Aliens trick) replaces the constraint with a penalty λ per transition, reducing to an unconstrained DP. Binary search on λ finds the value that yields exactly k transitions. This combines binary search on the answer with a D&C DP, giving total cost $\mathcal{O}(n \log n \log(\text{range}))$.

10.5.3 Convex hull trick

When $w(j,i)$ is linear in i (i.e., $w(j,i) = a_j \cdot i + b_j$), the DP can be accelerated to $\mathcal{O}(n \log n)$ or $\mathcal{O}(n)$ via the convex hull trick. This is a special case of D&C DP where the cost structure is more constrained.

10.6 Chapter Summary

Divide-and-conquer DP optimisation applies to 1D/1D dynamic programs whose cost function satisfies the quadrangle inequality. The monotonicity of the optimal decision pointer (Theorem 10.3) enables a divide-and-conquer recurrence that computes dp in $\mathcal{O}(n \log n)$ rather than $\mathcal{O}(n^2)$. The technique is the right tool whenever the cost function is convex in a suitable sense; verifying the QI is the key step in pattern identification.

Part III

Practice and Pitfalls

Chapter 11

Common Bugs and Pitfalls

Binary search is famous for off-by-one errors. The folklore is that only $\sim 10\%$ of programmers can write a correct binary search from scratch on the first try, and the figure is not much better among competitive programmers. This chapter catalogues the bugs that cause otherwise-correct solutions to fail on edge cases, with bad/good code comparisons for each. Internalising these patterns is the single most valuable defensive skill the contestant can develop.

11.1 Bug 1: Integer Overflow in Midpoint

Warning

Symptom: the solution crashes or returns wrong answers on inputs where the search range is large (e.g., $[0, 2 \cdot 10^9]$ on a 32-bit `int`).

```
// BAD: lo + hi can overflow int when both are near 1e9.
int mid = (lo + hi) / 2;
```

Listing 11.1: Bad: overflow in midpoint.

```
// GOOD: difference cannot overflow (it is at most the search range).
int mid = lo + (hi - lo) / 2;
```

Listing 11.2: Good: overflow-safe midpoint.

The overflow is silent in C++ (signed overflow is undefined behaviour, but in practice wraps around to a negative value), producing a negative `mid` that then indexes out of bounds. The fix is the identity $\mathsf{mid} = \mathsf{lo} + (\mathsf{hi} - \mathsf{lo})/2$, which cannot overflow because $\mathsf{hi} - \mathsf{lo}$ is bounded by the search range size. This form is mandatory for all integer binary searches with `int`; using `long long` also avoids the issue but is wasteful if indices fit in `int`.

11.2 Bug 2: Infinite Loop with Closed-Interval Mid

Warning

Symptom: the solution times out on a single test case; no progress is being made.

```
// BAD: if hi = lo + 1 and we set lo = mid, mid = lo, no progress.
while (lo < hi) {
    int mid = (lo + hi) / 2;
    if (pred(mid)) hi = mid;
```

```
    else            lo = mid;   // BUG: should be lo = mid + 1
}
```

Listing 11.3: Bad: infinite loop when hi = lo + 1.

```
// GOOD: when lo = mid, use ceiling midpoint; when hi = mid, use floor.
while (lo < hi) {
    int mid = lo + (hi - lo) / 2;        // floor: safe with hi = mid
    if (pred(mid)) hi = mid;
    else           lo = mid + 1;         // +1 guarantees progress
}
// OR (alternative form):
while (lo < hi) {
    int mid = lo + (hi - lo + 1) / 2;    // ceiling: safe with lo = mid
    if (pred(mid)) hi = mid - 1;
    else           lo = mid;
}
```

Listing 11.4: Good: paired updates that guarantee progress.

The infinite loop arises because when $\mathsf{hi} = \mathsf{lo}+1$, the floor midpoint $\lfloor(\mathsf{lo}+\mathsf{hi})/2\rfloor$ equals lo, and the update `lo = mid` leaves `lo` unchanged. The fix is to use `lo = mid + 1` (which guarantees progress) or, if the update must be `lo = mid` (because `mid` might itself be the answer), to use the ceiling midpoint `mid = lo + (hi - lo + 1)/ 2`.

The open-interval convention (Algorithm 1) sidesteps this entirely: with loop condition `hi - lo > 1` and floor midpoint, both `lo = mid` and `hi = mid` always make progress because mid is strictly between lo and hi.

11.3 Bug 3: Wrong Sentinel at Boundaries

Warning

Symptom: the solution returns the wrong answer when the target is absent, or when the target is the smallest/largest element.

```
// BAD: returns garbage if no element satisfies pred.
int lo = 0, hi = n - 1;
while (lo < hi) {
    int mid = (lo + hi) / 2;
    if (pred(mid)) hi = mid;
    else           lo = mid + 1;
}
return lo;  // What if pred is false everywhere? We return n-1, wrong.
```

Listing 11.5: Bad: no virtual boundary; fails when target is absent.

```
// GOOD: hi = n is the virtual element on which pred is true.
int lo = -1, hi = n;  // lo = -1 is virtual false; hi = n is virtual true
while (hi - lo > 1) {
    int mid = lo + (hi - lo) / 2;
    if (pred(mid)) hi = mid;
    else           lo = mid;
}
return hi;  // Smallest index with pred(i) = true; n if none.
```

Listing 11.6: Good: virtual boundary on the right.

The virtual-boundary trick is what the `hi++` line in Listing 1.1 accomplishes. Without it, the caller must check separately whether the returned index actually satisfies the predicate—an easy step to forget.

11.4 Bug 4: Wrong Predicate Direction

Warning

Symptom: the solution converges to the wrong endpoint of the search range.

```
// BAD: We want smallest x with feasible(x), but updates go wrong way.
while (lo < hi) {
    int mid = (lo + hi) / 2;
    if (feasible(mid)) lo = mid;      // should be hi = mid
    else               hi = mid + 1; // should be lo = mid + 1
}
return lo;
```

Listing 11.7: Bad: predicate direction reversed.

The fix is to write the invariant explicitly before the loop: **"lo is infeasible (or below the answer); hi is feasible (or above the answer)"**. Then the updates follow mechanically:

- If pred(mid) is true, mid is feasible, so set hi ← mid (the answer is at most mid).
- If pred(mid) is false, mid is infeasible, so set lo ← mid (the answer is at least mid).

The returned value is hi (the smallest feasible). Reversing the updates yields the *largest* infeasible, which is usually not what was intended.

11.5 Bug 5: Off-By-One in Closed-Interval Update

Warning

Symptom: the solution returns the index just before or just after the correct answer.

```
// BAD: returns smallest i with a[i] >= v, but off by one in some cases.
while (lo < hi) {
    int mid = (lo + hi) / 2;
    if (a[mid] >= v) hi = mid;        // correct
    else             lo = mid;        // BUG: should be lo = mid + 1
}
return lo;  // may be off by 1
```

Listing 11.8: Bad: missing +1 in lower bound update.

The fix is to use `lo = mid + 1` when `mid` is known to be infeasible. The off-by-one is particularly insidious because the solution may pass simple tests but fail on edge cases (e.g., target smaller than the smallest element).

11.6 Bug 6: Real-Valued EPS Misuse

Warning

Symptom: the solution gives wrong answer by a small fraction (10^{-7} instead of 10^{-6} required), or times out on real-valued problems.

```
// BAD: may not converge if range is huge.
double lo = 0, hi = 1e18;
while (hi - lo > 1e-9) {  // 1e-9 too small for 1e18 range
    double mid = (lo + hi) / 2;
```

```
    if (pred(mid)) hi = mid;
    else           lo = mid;
}
return (lo + hi) / 2;
```

Listing 11.9: Bad: EPS too small; may not converge.

```
// GOOD: fixed iteration count; converges to floating-point precision.
double lo = 0, hi = 1e18;
for (int i = 0; i < 200; i++) {  // 200 iters reduces range by 2^200
    double mid = (lo + hi) / 2;
    if (pred(mid)) hi = mid;
    else           lo = mid;
}
return (lo + hi) / 2;
```

Listing 11.10: Good: fixed iteration count.

The EPS-based termination is fragile for two reasons: (1) if the range is too wide, the loop never terminates because the difference never drops below ε within floating-point precision; (2) if the predicate has discontinuities, the loop may oscillate near the discontinuity. The fixed-iteration approach is uniformly safe and is the recommended default.

11.7 Bug 7: Returning the Wrong Variable

Warning

Symptom: the solution is off by one at the boundary.

In the open-interval convention, the loop terminates when $\mathsf{hi} - \mathsf{lo} = 1$, and the answer is hi (the smallest feasible). A common mistake is to return lo instead, which is the largest *infeasible* element—off by one from the correct answer.

```
while (hi - lo > 1) {
    int mid = lo + (hi - lo) / 2;
    if (pred(mid)) hi = mid;
    else           lo = mid;
}
return lo;  // BUG: returns largest infeasible; should be hi.
```

Listing 11.11: Bad: returning lo instead of hi.

The fix is to memorise the convention: **open-interval BS returns `hi`**. If the loop direction is reversed (largest feasible), return `lo` instead.

11.8 Bug 8: Predicate Mutates State

Warning

Symptom: the solution gives inconsistent answers across test cases, or fails on the second test case but passes the first.

```
vector<int> cache;  // mutated inside predicate
bool pred(int x) {
    cache.push_back(x);  // BUG: state grows, predicate changes meaning
    return /* ... */;
}
```

Listing 11.12: Bad: predicate modifies global state.

The predicate must be a pure function of x and the (read-only) input. Any mutation of global state breaks the monotonicity assumption and causes the search to diverge. The fix is to make the predicate a pure function: pass all needed state as const-reference parameters and never mutate them.

11.9 Bug 9: Closed-Interval with Empty Range

Warning

Symptom: the solution crashes on empty input or out-of-range queries.

```
// BAD: crashes on empty array (mid computed on negative range).
int find_first(const vector<int>& a, int v) {
    int lo = 0, hi = a.size() - 1;
    while (lo < hi) {
        int mid = (lo + hi) / 2;
        if (a[mid] >= v) hi = mid;
        else             lo = mid + 1;
    }
    return lo;
}
```

Listing 11.13: Bad: no handling of empty array.

When `a` is empty, `hi = -1`, and the loop body accesses `a[mid]` with a negative index. The fix is to handle the empty case explicitly, or to use the open-interval convention with virtual sentinels (which handles emptiness automatically).

11.10 Bug 10: Wrong Search Range

Warning

Symptom: the solution returns an answer outside the valid range, or fails to find an answer that exists.

```
// BAD: max subarray sum can exceed sum of positives if negatives present.
long long lo = 0, hi = 0;
for (int v : a) hi += max(0, v);  // BUG: should be sum of all (or max element)
```

Listing 11.14: Bad: range too narrow; misses the answer.

The search range must contain the answer with margin. For "minimise the maximum subarray sum", the lower bound is $\max_i a_i$ (each subarray contains at least one element) and the upper bound is $\sum_i a_i$ (one subarray suffices). Using $\sum \max(0, a_i)$ as the upper bound is wrong if the array has negatives.

11.11 A Pre-Submission Checklist

Before submitting any binary search solution, verify:

C1. Midpoint uses `lo + (hi - lo)/ 2`, not `(lo + hi)/ 2`.

C2. Loop condition and updates are paired correctly (open-interval convention: `hi - lo > 1` with `lo = mid` and `hi = mid`).

C3. Virtual sentinels are present on both sides of the search range.

C4. The predicate is a pure function (no global mutation).

C5. The return value is `hi` (open-interval, smallest feasible) or `lo` (reversed direction, largest feasible).

C6. Empty input is handled.

C7. The search range contains the answer with margin.

C8. For real-valued BS, fixed iteration count (not EPS) is used.

11.12 Chapter Summary

The ten bugs catalogued in this chapter account for the vast majority of binary search failures. They share a common root cause: the loop invariant was not written down explicitly before coding, so the update direction, return value, or boundary handling was guessed rather than derived. The single most effective defence is to write the invariant—"`lo` is infeasible, `hi` is feasible"—as a comment at the top of the loop, and to derive the updates and return value mechanically from it. The pre-submission checklist of §11.11 condenses the defensive practice into eight items; running through it before every submission eliminates the great majority of binary search bugs.

Chapter 12

Additional Worked Examples

This chapter presents four additional fully worked problems not covered in the pattern chapters. Each illustrates a different combination of patterns and a different failure mode that the solver should be aware of.

12.1 Example 1: Koko Eating Bananas (LeetCode 875)

Problem 12.1. Koko loves to eat bananas. There are n piles of bananas; the i-th pile has `piles[i]` bananas. The guards have gone and will come back in h hours. Koko can decide her per-hour eating speed k. Each hour, she chooses some pile and eats up to k bananas from it; if the pile has less than k bananas, she eats all of them and will not eat any other pile during this hour. Koko likes to eat slowly but wants to finish all the bananas before the guards return. Return the minimum integer k such that she can eat all the bananas within h hours.

Pattern identification. "Minimum integer k such that she can eat all the bananas" is the textbook BSA cue. The predicate is "can all piles be finished in $\leq h$ hours at speed k?"; it is monotone increasing in k (faster eating finishes earlier). Q1–Q4 all affirm.

Predicate.

$$\Phi(k) := \left(\sum_i \left\lceil \frac{\text{piles}[i]}{k} \right\rceil \leq h \right).$$

Monotone increasing in k.

Search range. $L = 1$ (slowest), $R = \max_i \text{piles}[i]$ (fastest, finishes in $n \leq h$ hours).

Complexity. $\mathcal{O}(n \log(\max \text{piles}))$.

```cpp
class Solution {
public:
    int minEatingSpeed(vector<int>& piles, int h) {
        long long lo = 1, hi = *max_element(piles.begin(), piles.end());
        while (hi - lo > 1) {
            long long mid = lo + (hi - lo) / 2;
            if (can_finish(piles, h, mid)) hi = mid;
            else                           lo = mid;
        }
        return (int)hi;
    }
private:
    bool can_finish(const vector<int>& piles, int h, long long k) {
        long long hours = 0;
        for (int p : piles) {
            hours += (p + k - 1) / k;  // ceil(p / k) without floats
            if (hours > h) return false;
        }
        return hours <=> h;
```

```
    }
};
```

Listing 12.1: Solution to LeetCode 875.

12.2 Example 2: Capacity To Ship Packages (LeetCode 1011)

Problem 12.2. A conveyor belt has packages that must be shipped from one port to another within D days. The i-th package has weight `weights[i]`. Each day, we load the ship with packages on the conveyor belt (in the order given by `weights`). We may not load more weight than the maximum weight capacity of the ship. Return the least weight capacity that will result in all the packages being shipped within D days.

Pattern identification. "Least weight capacity such that shipping finishes within D days" is BSA. The predicate is "can the packages be shipped within D days at capacity C?"; monotone increasing in C (larger capacity = fewer days).

Predicate. Greedy: walk left to right, adding packages to the current day's load until the next would exceed C; then start a new day. $\Phi(C)$ is true iff the day count is $\leq D$.

Search range. $L = \max_i w_i$, $R = \sum_i w_i$.

Complexity. $\mathcal{O}(n \log(\sum w_i))$.

```
class Solution {
public:
    int shipWithinDays(vector<int>& weights, int days) {
        long long lo = 0, hi = 0;
        for (int w : weights) {
            lo = max(lo, (long long)w);
            hi += w;
        }
        while (hi - lo > 1) {
            long long mid = lo + (hi - lo) / 2;
            if (can_ship(weights, days, mid)) hi = mid;
            else                              lo = mid;
        }
        return (int)hi;
    }
private:
    bool can_ship(const vector<int>& w, int D, long long C) {
        int days = 1;
        long long cur = 0;
        for (int x : w) {
            if (cur + x > C) {
                days++;
                cur = x;
                if (days > D) return false;
            } else {
                cur += x;
            }
        }
        return true;
    }
};
```

Listing 12.2: Solution to LeetCode 1011.

12.3 Example 3: Pie (POJ 3122)

Problem 12.3. My birthday is coming up and traditionally I am serving pie. I have N friends, and each has a pie of known volume (the i-th pie has volume V_i). I want to give each friend (including myself) a single piece of pie of the same volume. What is the maximum possible volume of each piece?

Pattern identification. "Maximum possible volume" + "real-valued answer" (volume can be fractional) signals real-valued BSA. The predicate is "can $N+1$ pieces of volume $\geq v$ each be cut from the pies?"; monotone decreasing in v.

Predicate. For each pie of volume V_i, the number of pieces of volume v it can yield is $\lfloor V_i/v \rfloor$. Sum over all pies; predicate is true iff the sum is $\geq N+1$.

Search range. $L = 0$, $R = \max_i V_i$.

Complexity. $\mathcal{O}(N \cdot \text{iters})$, typically $\mathcal{O}(N \cdot 100)$.

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    cout << fixed << setprecision(4);
    int T; cin >> T;
    while (T--) {
        int N, F; cin >> N >> F;
        F++;  // include self
        vector<double> V(N);
        double maxV = 0;
        for (int i = 0; i < N; i++) {
            int r; cin >> r;
            V[i] = M_PI * r * r;
            maxV = max(maxV, V[i]);
        }
        double lo = 0, hi = maxV;
        for (int it = 0; it < 100; it++) {
            double mid = (lo + hi) / 2;
            long long cnt = 0;
            for (double v : V) cnt += (long long)(v / mid);
            if (cnt >= F) lo = mid;  // can serve F people at volume mid
            else          hi = mid;
        }
        cout << lo << "\n";
    }
    return 0;
}
```

Listing 12.3: Solution to POJ 3122 (Pie).

12.4 Example 4: Allocate Mailboxes (LeetCode 1478)

Problem 12.4. Given an array `houses` of n integers representing the positions of houses on a line, and an integer k representing the number of mailboxes to allocate, find the minimum total distance from each house to its nearest mailbox.

Pattern identification. The problem is a layered DP: $dp[i][j]$ = minimum total distance to serve houses $[1..i]$ with j mailboxes. The transition is

$$dp[i][j] = \min_{i'<i} \{dp[i'][j-1] + w(i'+1, i)\},$$

where $w(l, r)$ is the cost of serving houses $[l, r]$ with one mailbox (placed at the median). The cost function w is convex in (l, r) and satisfies the quadrangle inequality, so D&C DP applies. Alternatively, since k is small, the $\mathcal{O}(n^2 k)$ naive DP is acceptable.

Approach. Precompute $w(l, r)$ for all $l \le r$ via prefix sums in $\mathcal{O}(n^2)$. Then compute dp layer by layer; for each layer, use D&C DP to reduce from $\mathcal{O}(n^2)$ to $\mathcal{O}(n \log n)$.

Complexity. $\mathcal{O}(n^2 + nk \log n)$, or $\mathcal{O}(n^2 k)$ without D&C DP.

```
class Solution {
public:
    int minDistance(vector<int>& houses, int k) {
        int n = houses.size();
        sort(houses.begin(), houses.end());
        // Precompute cost[l][r] = sum of |houses[i] - median| for i in [l, r].
        vector<vector<int>> cost(n, vector<int>(n, 0));
        for (int l = 0; l < n; l++)
            for (int r = l; r < n; r++) {
                int m = (l + r) / 2;
                int c = 0;
                for (int i = l; i<=>r; i++)
                    c += abs(houses[i] - houses[m]);
                cost[l][r] = c;
            }
        // dp[j][i] = min cost to serve houses[0..i] with j mailboxes.
        vector<vector<int>> dp(k + 1, vector<int>(n, INT_MAX));
        for (int i = 0; i < n; i++) dp[1][i] = cost[0][i];
        for (int j = 2; j<=>k; j++) {
            // D&Q DP: solve(l, r, optL, optR).
            function<void(int,int,int,int)> solve =
                [&](int l, int r, int optL, int optR) {
                if (l > r) return;
                int m = (l + r) / 2;
                int best = optL;
                dp[j][m] = INT_MAX;
                for (int t = optL; t<=>min(optR, m - 1); t++) {
                    int val = dp[j-1][t] + cost[t+1][m];
                    if (val < dp[j][m]) {
                        dp[j][m] = val;
                        best = t;
                    }
                }
                solve(l, m - 1, optL, best);
                solve(m + 1, r, best, optR);
            };
            solve(0, n - 1, 0, n - 2);
        }
        return dp[k][n-1];
    }
};
```

Listing 12.4: Solution to LeetCode 1478 (Allocate Mailboxes).

12.5 Example 5: Magnetic Force Between Two Balls (LeetCode 1552)

Problem 12.5. Given n positions of baskets on a line and an integer m (number of balls), place m balls in the baskets such that the minimum magnetic force between any two balls is maximised. The magnetic force between two balls at positions x and y is $|x - y|$.

This is the LeetCode version of Aggressive Cows (§4.3). We omit the solution; the reader should implement it as an exercise using the template of Listing 4.1.

12.6 Example 6: Heaters (LeetCode 475)

Problem 12.6. Winter is coming! Your first job during the contest is to design a standard heater with a fixed warm radius to warm all the houses. Every house can be warmed as long as it is within the radius range of a heater. Given positions of `houses` and `heaters` on a horizontal line, return the minimum radius needed to warm all houses.

Pattern identification. "Minimum radius such that every house is within the radius of some heater" is BSA. The predicate "can all houses be covered at radius r?" is monotone increasing in r.

Predicate. For each house, find the nearest heater (via `lower_bound` on sorted `heaters`); the house is covered iff its distance to the nearest heater is $\leq r$. The predicate is true iff every house is covered.

Search range. $L = 0$, $R =$ (max distance between any house and any heater).

Complexity. $\mathcal{O}((n+m)\log(\text{range}))$ after sorting.

```cpp
class Solution {
public:
    int findRadius(vector<int>& houses, vector<int>& heaters) {
        sort(heaters.begin(), heaters.end());
        long long lo = 0, hi = 0;
        // Compute hi: max distance from any house to nearest heater.
        for (int h : houses) {
            auto it = lower_bound(heaters.begin(), heaters.end(), h);
            long long d = LLONG_MAX;
            if (it != heaters.end()) d = min(d, (long long)*it - h);
            if (it != heaters.begin()) d = min(d, (long long)h - *prev(it));
            hi = max(hi, d);
        }
        hi += 1;  // virtual infeasible (just above the answer)
        while (hi - lo > 1) {
            long long mid = lo + (hi - lo) / 2;
            if (can_cover(houses, heaters, mid)) lo = mid;  // lo feasible
            else                                 hi = mid;  // hi infeasible
        }
        return (int)lo;  // largest feasible
    }
private:
    bool can_cover(const vector<int>& houses, const vector<int>& heaters, long
        long r) {
        for (int h : houses) {
            auto it = lower_bound(heaters.begin(), heaters.end(), h - r);
            if (it == heaters.end() || *it > h + r) return false;
        }
        return true;
    }
};
```

Listing 12.5: Solution to LeetCode 475 (Heaters).

12.7 Chapter Summary

The six additional worked examples in this chapter illustrate the breadth of the BSA family: integer-speed eating (Koko), integer capacity shipping, real-valued volume cutting (Pie), D&C DP optimisation (Allocate Mailboxes), Aggressive Cows revisited (Magnetic Force), and predicate-evaluation-via-lower-bound (Heaters). Each problem fits the four-question procedure of §2.1, but the predicate designs and search ranges differ substantially. The skill of pattern

identification is the skill of recognising, across these surface differences, the common monotone-predicate-inversion structure.

Chapter A

Code Templates Reference

This appendix collects the reference templates from the main text into a single location for quick lookup. Each template is self-contained and uses only the standard C++ library.

A.1 Open-Interval Integer Binary Search

```cpp
// Returns the smallest x in [lo, hi] with p(x) == true.
// Pre: p(hi) is true; the function adds a virtual element at hi+1.
template <class Pred>
long long first_true(long long lo, long long hi, Pred p) {
    hi++;
    while (hi - lo > 1) {
        long long mid = lo + (hi - lo) / 2;
        if (p(mid)) hi = mid;
        else        lo = mid;
    }
    return hi;
}
```

Listing A.1: Open-interval binary search (integer).

A.2 Lower and Upper Bound

```cpp
int lower_bound_idx(const vector<int>& a, int v) {
    int lo = -1, hi = (int)a.size();
    while (hi - lo > 1) {
        int mid = lo + (hi - lo) / 2;
        if (a[mid] >= v) hi = mid;
        else             lo = mid;
    }
    return hi;
}

int upper_bound_idx(const vector<int>& a, int v) {
    int lo = -1, hi = (int)a.size();
    while (hi - lo > 1) {
        int mid = lo + (hi - lo) / 2;
        if (a[mid] > v) hi = mid;
        else            lo = mid;
    }
    return hi;
}
```

Listing A.2: Lower and upper bound implementations.

A.3 Binary Search on the Answer

```
// Find smallest X in [L, R] such that feasible(X) is true.
// Pre: feasible(R) is true; feasible(L - 1) is false.
template <class Feas>
long long bsa(long long L, long long R, Feas feasible) {
    long long lo = L - 1, hi = R;
    while (hi - lo > 1) {
        long long mid = lo + (hi - lo) / 2;
        if (feasible(mid)) hi = mid;
        else               lo = mid;
    }
    return hi;
}
```

Listing A.3: BSA template.

A.4 Real-Valued Binary Search

```
template <class Pred>
double first_true_real(double lo, double hi, Pred p, int iters = 100) {
    for (int i = 0; i < iters; i++) {
        double mid = (lo + hi) / 2.0;
        if (p(mid)) hi = mid;
        else        lo = mid;
    }
    return (lo + hi) / 2.0;
}
```

Listing A.4: Real-valued binary search.

A.5 Ternary Search

```
template <class F>
long long argmax_ternary(long long lo, long long hi, F f) {
    while (hi - lo > 2) {
        long long m1 = lo + (hi - lo) / 3;
        long long m2 = hi - (hi - lo) / 3;
        if (f(m1) < f(m2)) lo = m1;
        else               hi = m2;
    }
    long long best = lo;
    for (long long i = lo; i<=>hi; i++)
        if (f(i) > f(best)) best = i;
    return best;
}

template <class F>
double argmax_ternary_real(double lo, double hi, F f, int iters = 200) {
    for (int i = 0; i < iters; i++) {
        double m1 = lo + (hi - lo) / 3.0;
        double m2 = hi - (hi - lo) / 3.0;
        if (f(m1) < f(m2)) lo = m1;
        else               hi = m2;
    }
    return (lo + hi) / 2.0;
}
```

Listing A.5: Ternary search for the maximum of a unimodal function.

A.6 Parallel Binary Search Skeleton

```cpp
// generic_data_structure ds; with reset(), apply(event_t), check(query_t)
template <class DS, class Event, class Query, class Check>
vector<int> pbs(int Q, int T, DS ds, const vector<Event>& events,
                vector<Query>& queries, Check check) {
    vector<int> lo(Q, 0), hi(Q, T + 1), ans(Q, T + 1);
    while (true) {
        vector<vector<int>> at_mid(T + 2);
        bool any = false;
        for (int i = 0; i < Q; i++)
            if (hi[i] - lo[i] > 1) {
                any = true;
                at_mid[(lo[i] + hi[i]) / 2].push_back(i);
            }
        if (!any) break;
        ds.reset();
        for (int t = 1; t<=>T; t++) {
            ds.apply(events[t - 1]);
            for (int i : at_mid[t]) {
                if (check(ds, queries[i])) { hi[i] = t; ans[i] = t; }
                else                       lo[i] = t;
            }
        }
    }
    return ans;
}
```

Listing A.6: PBS skeleton.

A.7 Binary Lifting (LCA)

```cpp
struct LCA {
    int n, LOG;
    vector<vector<int>> up;
    vector<int> depth;
    const vector<vector<int>>& adj;
    LCA(int root, const vector<vector<int>>& g)
        : n(g.size()), LOG(0), adj(g) {
        while ((1 << LOG) < n) LOG++;
        up.assign(LOG, vector<int>(n));
        depth.assign(n, 0);
        dfs(root, root);
        for (int i = 1; i < LOG; i++)
            for (int v = 0; v < n; v++)
                up[i][v] = up[i-1][up[i-1][v]];
    }
    void dfs(int v, int p) {
        up[0][v] = p;
        for (int u : adj[v]) if (u != p) {
            depth[u] = depth[v] + 1;
            dfs(u, v);
        }
    }
    int kth_ancestor(int v, int k) const {
```

```
        for (int i = 0; i < LOG && k > 0; i++) {
            if (k & 1) v = up[i][v];
            k >>= 1;
        }
        return v;
    }
    int lca(int u, int v) const {
        if (depth[u] < depth[v]) swap(u, v);
        u = kth_ancestor(u, depth[u] - depth[v]);
        if (u == v) return u;
        for (int i = LOG - 1; i >= 0; i--)
            if (up[i][u] != up[i][v]) {
                u = up[i][u];
                v = up[i][v];
            }
        return up[0][u];
    }
    int dist(int u, int v) const {
        return depth[u] + depth[v] - 2 * depth[lca(u, v)];
    }
};
```

Listing A.7: LCA via binary lifting.

A.8 Divide and Conquer DP

```
// Computes dp[layer][i] for i in [l, r], given that the optimal
// decision pointer opt(i) lies in [optL, optR].
template <class CostFn>
void dcdp_layer(vector<vector<long long>>& dp, int layer,
                int l, int r, int optL, int optR, CostFn cost) {
    if (l > r) return;
    int m = (l + r) / 2;
    dp[layer][m] = LLONG_MAX;
    int best = optL;
    for (int j = optL; j<=>min(optR, m - 1); j++) {
        long long val = dp[layer - 1][j] + cost(j + 1, m);
        if (val < dp[layer][m]) {
            dp[layer][m] = val;
            best = j;
        }
    }
    dcdp_layer(dp, layer, l, m - 1, optL, best, cost);
    dcdp_layer(dp, layer, m + 1, r, best, optR, cost);
}
```

Listing A.8: D&C DP optimisation template.

A.9 Pre-Submission Checklist

C1. Midpoint: `lo + (hi - lo)/ 2`, not `(lo + hi)/ 2`.

C2. Loop condition and updates paired (open-interval: `hi - lo > 1`, `lo = mid`, `hi = mid`).

C3. Virtual sentinels on both sides.

C4. Predicate is pure (no global mutation).

C5. Return `hi` (open-interval smallest feasible) or `lo` (largest feasible in reversed direction).

C6. Empty input handled.

C7. Search range contains the answer with margin.

C8. Real-valued BS: fixed iteration count, not EPS.